05 · API documentation

Dubai Real Estate CRM · Version 1.2 · 11 October 2026 · Matches SRS v1.2 and database v1.2

The full contract is in `api/openapi.yaml` (OpenAPI 3.1): **287 endpoints, 109 data models**. In the running app the same contract is shown as interactive **Swagger UI at `/api/docs`**. This guide explains the rules every endpoint follows.

---

1. In simple words

The **screens** (web and phone) never touch the database. They ask the **API** for data and send it changes, like a waiter taking orders to the kitchen. **Outside systems** (Meta, WhatsApp, Property Finder, Bayut, Dubizzle, the general webhook) send new leads and message updates to special **webhook** addresses. Every request is checked: who are you, which company are you in, and are you allowed to do this?

```
Screens (Next.js) ──HTTPS──▶  /api/v1/*        ──▶ NestJS API ──▶ PostgreSQL
Clients, new
customers          ──HTTPS──▶  /api/v1/public/* (booking page, pricing, sign-up; no login)
Platform team      ──HTTPS──▶  /api/v1/platform/* (owner console, separate login)
Meta, WhatsApp, portals,
webhook senders,
Stripe             ──HTTPS──▶  /webhooks/*      ──▶ saved, then processed by the worker
```

2. Basics

ITEM	RULE
Base URL	`https://crm.example.ae/api/v1` · staging `https://staging.crm.example.ae/api/v1` · local `http://localhost:4000/api/v1`
Format	JSON, UTF-8. Field names in `camelCase`.
Times	ISO 8601 in UTC, e.g. `2026-10-11T08:30:00Z`. Screens convert to the user's (or client's) time zone.
Money	Decimal **string** with 2 places, e.g. `"3250000.00"`, plus a `currency` (ISO code). Never floating point.
IDs	UUID strings.
Versioning	`/api/v1`. Adding fields is not a breaking change; removing or renaming is, and needs `/api/v2`.

3. Login and access

1. The screen sends the user to the **Cognito** login page (OIDC + PKCE). Two-factor login is required.
2. Cognito returns an **access token** (valid 1 hour) and a refresh token.
3. Every API call sends `Authorization: Bearer <access token>`.
4. For every request the API runs, in order: **token check → tenant → permission → data scope → input validation**.
5. `GET /me` returns the user, tenant, permissions and data scope; the screens use it to show or hide menus and buttons. The server still checks every action itself.

DATA SCOPE	SEES
own	Own records, plus leads shared with the user
team	Records of the teams the user leads or manages
division	Records of the user's divisions
company	Everything in the tenant

A record outside the user's scope returns **404** (not 403), so its existence is not revealed.

4. Lists, filters and sorting

- Lists use **cursor paging**: `?limit=50&cursor=…`. The response is `{ "data": [...], "page": { "nextCursor": "…", "hasMore": true } }`. Add `total=true` only when a count is needed.
- Filters are query parameters, e.g. `GET /leads?tab=overdue&projectId=…&createdFrom=2026-10-01`.
- Sorting: `sort=-score,nextFollowUpAt` (minus = descending). Leads default to best score first.
- Saved filters (`/saved-filters`) store the same parameters by name.

5. Creating and changing data safely

HEADER	USE
`Idempotency-Key`	On important POSTs (create lead, send message, record payment). Retrying with the same key returns the first result, so a double-click or network retry never creates duplicates.
`If-Match` / `version`	Optimistic locking on leads and financial documents. If someone else saved first, the API returns **409 VERSION_CONFLICT** and the screen asks the user to reload.

Business actions have their own endpoints so the rules and history are always applied, for example `POST /leads/{id}/stage`, `POST /leads/{id}/assign`, `POST /units/{id}/status`, `POST /finance-documents/{id}/approve`.

6. Errors

All errors use **problem details** (`application/problem+json`):

```
{
  "type": "https://crm.example.ae/errors/lead-exists",
  "title": "Lead already exists",
  "status": 409,
  "code": "LEAD_EXISTS",
  "detail": "This lead already exists, owned by Sara Ali.",
  "requestId": "req_7f3a…"
}
```

HTTP	MEANING
400	Invalid input (`errors[]` lists each field)
401	Not logged in or token expired
403	Logged in but not allowed (permission)
404	Not found, or outside your scope
402	Company is read-only: subscription unpaid after the 14-day grace (`SUBSCRIPTION_READ_ONLY`). Reading and exporting still work.
409	Conflicts with the current state
422	A business rule failed
429	Too many requests (see `Retry-After`)

Business error codes (the `code` field; screens show a friendly message for each):

CODE	WHEN
LEAD_EXISTS	New lead's phone/email belongs to an existing lead (re-enquiry rule)
REASON_REQUIRED	Lost/Invalid stage or reassignment without a reason
USER_NOT_RECEIVING_LEADS	Assigning to a user whose lead assignment is switched off
USER_HAS_OPEN_LEADS	Deactivating a user who still owns open leads
VERSION_CONFLICT	Someone else saved the record first
OUTCOME_REQUIRED / NEXT_ACTION_REQUIRED	Completing a call without an outcome or next action
AGENT_BUSY / SLOT_TAKEN	Meeting overlaps the agent's calendar
SLUG_TAKEN	Booking-page link name already used
OPTED_OUT	Message to a lead who opted out of that channel

CODE	WHEN
TEMPLATE_REQUIRED	WhatsApp outside the 24-hour reply window needs an approved template
TEST_REQUIRED / SELF_APPROVAL	Campaign submitted without a test / creator approving their own campaign
UNIT_ALREADY_BOOKED	Unit is held by another deal
NOT_DRAFT	Editing an approved financial document
AMOUNT_EXCEEDS_BALANCE / CURRENCY_MISMATCH	Invalid payment
ALREADY_CHECKED_IN / LOCATION_REQUIRED / OUTSIDE_OFFICE_RADIUS	Attendance check-in rules
LEAVE_OVERLAP / SELF_REVIEW	Leave overlaps approved leave / approving your own request
ROLE_IN_USE	Deleting a role that users still have
FILE_TYPE_NOT_ALLOWED / FILE_TOO_LARGE	Upload rules (max 25 MB)
SUBSCRIPTION_READ_ONLY	Any change while the company is unpaid after the grace period (HTTP 402)
TENANT_SUSPENDED	Company suspended by the platform team (HTTP 403)
SEAT_LIMIT_REACHED	Adding or reactivating a user when all paid seats are used
SEATS_BELOW_ACTIVE_USERS	Reducing seats below the number of active users
PLAN_NOT_ALLOWED / PLAN_NOT_SELF_SERVICE	Plan does not fit the company type / Enterprise must go through sales
LICENCE_REQUIRED	Sign-up without trade licence + RERA ORN (brokerage) or RERA BRN (agent)
EMAIL_ALREADY_REGISTERED / CODE_INVALID / CODE_EXPIRED / EMAIL_NOT_VERIFIED / CARD_REQUIRED	Sign-up steps

7. Files

1. `POST /files/uploads` → returns a 10-minute upload link.
2. The screen uploads the file straight to S3 with `PUT`.
3. `POST /files/{id}/complete`, then link the file, e.g. `POST /leads/{id}/attachments`.
4. Downloads: `GET /files/{id}/download` returns a 5-minute link, only for files the user may see.

8. Inbound webhooks (how outside systems talk to us)

SOURCE	ADDRESS	SECURITY
Meta lead ads	`POST /webhooks/meta` (+ `GET` verification)	`X-Hub-Signature-256`
WhatsApp	`POST /webhooks/whatsapp` (+ `GET` verification)	`X-Hub-Signature-256`
Property Finder, Bayut, Dubizzle	`POST /webhooks/portals/{provider}`	Shared secret per account
General webhook (website, Google Sheets, Excel via Power Automate, Make.com, any tool)	`POST /webhooks/inbound/{token}`	`X-Signature`: HMAC-SHA256 of the body with the source secret
Portal lead emails	`POST /webhooks/email-inbound` (via AWS SES/SNS)	SNS signature
Google / Microsoft calendars	`POST /webhooks/calendar/{provider}`	Channel token

Every webhook is **saved first and answered within 1 second**, then processed in the background. The same event ID is never processed twice. Failures are retried 5 times, then shown on the Integrations screen with Retry.

General webhook example (e.g. from a website form or a Google Sheet row):

```
POST /webhooks/inbound/whk_3f9c2e…
Content-Type: application/json
X-Signature: sha256=5d41402abc4b2a76b9719d911017c592…

{ "name": "Priya Sharma", "mobile": "+971559871122", "email": "priya@mail.com",
  "interested_in": "Sobha Hartland", "submission_id": "row-1842" }
```

The source's **field mapping** (set by Admin) turns `name`, `mobile`, … into CRM fields. `submission_id` prevents double imports. A known phone or email becomes a **re-enquiry** on the existing lead.

9. Public booking page (no login)

- `GET /public/booking/{slug}`: the agent's free slots in the client's time zone.
- `POST /public/booking/{slug}`: books the slot, creates the Google Meet / Teams link and confirmation, and links the client to an existing lead (re-enquiry) or a new lead owned by that agent.
- Protected by rate limits and a bot check (Cloudflare Turnstile).

10. Limits

WHO	LIMIT
Signed-in user	600 requests per minute
Public booking page	30 requests per minute per IP
Public sign-up and contact sales	10 requests per minute per IP, plus bot check
Webhooks	1,200 requests per minute per source
Bulk lead actions	500 leads per request
Uploads	25 MB per file

10a. Subscriptions, billing and owner console

ITEM	RULE
Plans	Solo (1 agent), Team (brokerage), Enterprise (through sales). AED 99 per user per month, excluding 5% VAT; monthly or yearly.
Sign-up	`POST /public/signup` → `verify-email` → card saved with Stripe → `complete`. A card is required; the 14-day trial starts at once.
Licence	Brokerage: trade licence + RERA ORN. Individual agent: RERA BRN. Checked by the platform team; WhatsApp is connected only after verification.
Seats	Active users can never exceed paid seats (enforced by the database). Invited users don't use a seat until they accept.
Payment	Card through Stripe (automatic) or bank transfer (customer submits reference; platform team confirms).
Payment failed	Status `past_due`; 14 days' grace with reminders; then `read_only` (402 on every change). Paying reactivates at once. Data is never deleted while unpaid.
Cancel	At the end of the paid period, no refund for the started period. Data can be exported for 90 days, then deleted.
Owner console	`/platform/*`: separate Cognito user pool with two-factor login, own database role (`crm_platform`). Roles: owner, support, billing. Every action is in the platform audit log. Cannot read a company's leads or messages.
Stripe	`POST /webhooks/stripe` (signature checked, stored first, processed by the worker). Card details never reach our servers.

11a. Every SRS feature and the APIs that deliver it

SRS FEATURE (V1.2)	API CALLS (IN ORDER)
Login and permissions	Cognito login → `GET /me`
Global search	`GET /search?q=`
Lead tabs, filters, saved filters, columns	`GET /leads?tab=…&…` · `GET/POST/DELETE /saved-filters`
New lead by hand (with "already exists" warning)	`GET /leads/check-existing` → `POST /leads` (409 LEAD_EXISTS → open existing)
Leads from Meta / portals / webhook / WhatsApp	`POST /webhooks/meta` · `/webhooks/portals/{provider}` · `/webhooks/inbound/{token}` · `/webhooks/whatsapp` · `/webhooks/email-inbound` (worker creates lead or re-enquiry)
Excel / CSV import	`POST /files/uploads` → PUT file → `POST /leads/imports` (preview) → `POST /leads/imports/{id}/confirm` → `GET /leads/imports/{id}`
Re-enquiry to previous agent	Automatic in intake; visible in `GET /leads/{id}/inquiries` and `GET /leads/{id}/timeline`; agent gets a notification
Automatic assignment	Automatic on create; rules via `GET/POST/PATCH/DELETE /assignment-rules`, `PUT /assignment-rules/order`
10-minute rule → next agent → Admin	Automatic (worker); any of `POST /leads/{id}/notes`, `/calls`, `/stage`, `/messages`, `POST /tasks/{id}/complete` stops the timer; moves show in timeline
Manual assign / reassign (reason)	`POST /leads/{id}/assign` · bulk `POST /leads/bulk`
Admin: switch lead assignment on/off	`PUT /users/{id}/receives-leads` · `GET /users/{id}/workload`
Lead page and timeline	`GET /leads/{id}` · `GET /leads/{id}/timeline` · `GET /leads/{id}/inquiries`
Note, call, stage, priority, attach	`POST /leads/{id}/notes` · `/calls` · `/stage` · `/priority` · `/attachments`
Share lead (view / edit)	`GET/POST /leads/{id}/shares` · `DELETE /leads/{id}/shares/{userId}` · Shared tab `GET /leads?tab=shared`
Projects of interest	`PUT /leads/{id}/projects`
Delete / restore	`DELETE /leads/{id}` · `POST /leads/{id}/restore`
Tasks, calendar views, reminders, overdue	`GET/POST/PATCH /tasks` · `POST /tasks/{id}/complete` · `/cancel` (reminders and overdue alerts by worker → `GET /notifications`)
Meetings and viewings (Meet / Teams), no double booking	`GET /availability` → `POST /meetings` (409 AGENT_BUSY) · `PATCH /meetings/{id}` · `/cancel` · `/complete`
Client booking page	`GET/PUT /booking-pages/me` · public `GET/POST /public/booking/{slug}`

SRS FEATURE (V1.2)	API CALLS (IN ORDER)
Calendar sync	`POST /integrations` (google_workspace / microsoft_365, user scope) · `POST /webhooks/calendar/{provider}`
Team: users, hierarchy, roles and permissions	`/users` · `/users/{id}/deactivate` · `/teams` · `/divisions` · `/roles` · `GET /permissions`
WhatsApp and email one-to-one	`GET /conversations` · `GET /conversations/{id}/messages` · `POST /leads/{id}/messages` · `POST /conversations/{id}/read`
Templates	`/message-templates` · `POST /message-templates/{id}/submit`
Bulk campaigns with approval	`POST /message-campaigns` → `/preview-audience` → `/test` → `/submit` → `/approve` or `/reject` → `GET /message-campaigns/{id}/report`
Consent and opt-out	`GET/POST /leads/{id}/consents` (opt-out replies recorded automatically by webhook)
Developers, projects, buildings, units	`/developers` · `/projects` · `/projects/{id}/buildings` · `/units` · `/units/{id}/attachments`
Unit status / price with history	`POST /units/{id}/status` · `GET /units/{id}/history`
Deal: offer → EOI → booking → sold	`POST /deals` · `POST /deals/{id}/stage` (409 UNIT_ALREADY_BOOKED) · `/deals/{id}/attachments`
Invoices and purchase orders	`/counterparties` · `POST /finance-documents` → `PATCH` (draft) → `/approve` → `/send` · `/cancel` · `/pdf` · `/history` · `/attachments`
Payments and refunds	`GET/POST /finance-documents/{id}/payments`
Commissions	`/commissions` · `/approve` · `/mark-paid` · `/commissions/{id}/attachments`
Attendance check-in / out with photo	`POST /files/uploads` (photo) → `POST /attendance/check-in` · `/check-out` · `GET /attendance/records`
Corrections (original kept)	`POST /attendance/records/{id}/corrections` · `GET /attendance/corrections` · `POST /attendance/corrections/{id}/review`
Leave	`/leave-requests` · `/review` · `/cancel` · `/leave-types` · `/holidays` · `/attendance-policies`
Dashboard with drill-down	`GET /dashboard?…` → each bucket's `drillDown` filters → `GET /leads?…`
Reports and exports (Admin only)	`GET /reports` · `GET /reports/{key}` · `POST /reports/{key}/exports` · `GET /exports/{id}` · export log `GET /exports`
Ad spend, events	`/campaigns` · `GET/PUT /campaigns/{id}/spend` · `/events`
Meta reconciliation	`POST /integrations/{id}/reconciliations` · `GET /reconciliations`
Integrations screen	`GET/POST /integrations` · `/test` · `/retry` · `DELETE` · `/sync-runs` · `/events` · `POST /webhook-events/{id}/retry`
General webhook setup	`POST /webhook-sources` · `PUT /webhook-sources/{id}/mapping` · `/rotate-secret`
Settings	`/settings` · `/lead-stages` · `/reasons` · `/lead-sources` · `/currencies` · `GET/PATCH /organization`
Notifications and push	`GET /notifications` · `/read` · `/read-all` · `POST/DELETE /push-devices`

SRS FEATURE (V1.2)	API CALLS (IN ORDER)
Audit log	`GET /audit-logs`
Pricing page and sign-up with free trial	`GET /public/plans` → `POST /public/signup` → `/verify-email` → Stripe card → `/complete` · Enterprise: `POST /public/sales-enquiries`
Plan, seats, billing cycle, payment method	`GET /billing/subscription` · `/change-plan` · `PUT /billing/subscription/seats` · `/payment-method` · `POST /billing/card-setup`
Subscription invoices and payments	`GET /billing/invoices` · `/{id}` · `/{id}/pdf` · `POST /billing/invoices/{id}/bank-transfers` · `GET /billing/payments` · Stripe `POST /webhooks/stripe`
Payment failure → grace → read-only	Automatic (Stripe webhook + worker); visible in `GET /billing/history`; changes return 402
Cancel and data export	`POST /billing/subscription/cancel` · `/resume` · `POST /billing/data-export`
Owner console	`GET /platform/metrics` · `/platform/tenants` · `/verify` · `/suspend` · `/extend` · `PATCH /platform/subscriptions/{id}` · `/platform/bank-transfers` → `/confirm` · `/platform/plans` · `/platform/settings` · `/platform/sales-enquiries` → `/create-company` · `/platform/audit-logs`

11. Endpoints by section

SECTION	ENDPOINTS	PATHS
Session	2	me, search
Organization	25	divisions, organization, permissions, roles, teams, users
Settings	19	currencies, lead-sources, lead-stages, reasons, settings
Leads	27	leads, saved-filters
Assignment	5	assignment-rules
Marketing	11	campaigns, events
Tasks	5	tasks
Meetings	9	availability, booking-pages, meetings
Public booking	2	public/booking
Messaging	23	conversations, leads, message-campaigns, message-templates
Projects	22	buildings, developers, projects, units
Deals	7	deals
Finance	23	commissions, counterparties, finance-documents
Attendance	22	attendance, attendance-policies, holidays, leave-requests, leave-types
Dashboard	1	dashboard
Reports	7	exports, integrations, reconciliations, reports
Integrations	14	integrations, oauth, webhook-events, webhook-sources
Files	4	attachments, files
Notifications	5	notifications, push-devices
Audit	1	audit-logs
Public sign-up	6	public/plans, public/sales-enquiries, public/signup
Subscription & billing	15	billing/card-setup, billing/data-export, billing/history, billing/invoices, billing/payments, billing/subscription, billing/usage
Owner console	23	platform/audit-logs, platform/bank-transfers, platform/billing-invoices, platform/billing-payments, platform/me, platform/metrics, platform/plans, platform/sales-enquiries, platform/settings, platform/subscriptions, platform/tenants
Webhooks (inbound)	9	webhooks

Full details of every endpoint, field and response are in `api/openapi.yaml` and the API reference PDF.

12. How Swagger is generated and kept correct

1. **Contract first:** `api/openapi.yaml` is the agreed contract; this document and the reference are generated from it.
2. In the code, each NestJS controller uses `@nestjs/swagger` decorators; the app serves Swagger UI at `/api/docs` (disabled in production, or behind admin login).
3. **CI check:** on every pull request the build generates the spec from the code and compares it with `api/openapi.yaml`. Any difference fails the build until the contract is updated and approved. The screens' TypeScript API client is generated from the same file, so frontend and backend can never drift apart.

API reference

Every endpoint from `api/openapi.yaml`. ***** = required. The chip on the right is the permission needed.

Session 2 · Organization 25 · Settings 19 · Leads 27 · Assignment 5 · Marketing 11 · Tasks 5 · Meetings 9 · Public booking 2 · Messaging 23 · Projects 22 · Deals 7 · Finance 23 · Attendance 22 · Dashboard 1 · Reports 7 · Integrations 14 · Files 4 · Notifications 5 · Audit 1 · Public sign-up 6 · Subscription & billing 15 · Owner console 23 · Webhooks (inbound) 9 · Data models 109

Session

GET `/me` Get current user — `(any signed-in user)`

Who I am, my tenant, my permissions and data scope. The screens use this to show or hide menu items and buttons.

200 OK → Me
400 401 403 429 standard errors

GET `/search` Global search — `(any signed-in user)`

Search box on every screen. Results are limited to the user's scope.

PARAMETER	IN	TYPE	NOTES
`q` *	query	string	Name, phone or email (min 2 characters)
`limit`	query	integer	

200 OK → SearchResult
400 401 403 429 standard errors

Organization

GET `/organization` Get organization profile — `(any signed-in user)`

200 OK → Tenant
400 401 403 429 standard errors

PATCH `/organization` Update organization profile — `settings.manage`

Body: Tenant

200 OK → Tenant
400 401 403 429 standard errors

GET `/users` List users — `users.manage`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`status`	query	`invited` | `active` | `inactive`	
`roleId`	query	string (uuid)	
`teamId`	query	string (uuid)	
`q`	query	string	Search name or email

200 OK → { data: User[], page: PageInfo }
400 401 403 429 standard errors

POST `/users` Create user — `users.manage`

422 SEAT_LIMIT_REACHED when all paid seats are used (invited users do not use a seat until they accept).

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: UserCreate

201 OK → User
400 401 403 409 422 429 standard errors

GET `/users/{id}` Get user — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → User
400 401 403 404 429 standard errors

PATCH `/users/{id}` Update user — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: User

200 OK → User
400 401 403 404 409 422 429 standard errors

PUT `/users/{id}/receives-leads` Turn lead assignment on or off — `users.lead_assignment`

Admin switch. When off, the user keeps existing leads but gets no new ones.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`receivesLeads` *	boolean	

200 OK → User
400 401 403 404 429 standard errors

POST `/users/{id}/deactivate` Deactivate user — `users.manage`

422 USER_HAS_OPEN_LEADS unless reassignToUserId is given; leads are then moved and each move is logged.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`reassignToUserId`	string (uuid)	

200 OK → User
400 401 403 404 422 429 standard errors

POST `/users/{id}/reactivate` Reactivate user — `users.manage`

422 SEAT_LIMIT_REACHED when all paid seats are used (invited users do not use a seat until they accept).

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → User
400 401 403 404 422 429 standard errors

GET `/roles` List roles — `users.manage`

200 OK → { data: Role[] }
400 401 403 429 standard errors

POST `/roles` Create role — `users.manage`

Custom roles such as Sales, Basic or HR.

Body: Role

201 OK → Role
400 401 403 409 429 standard errors

PATCH `/roles/{id}` Update role — `users.manage`

Built-in role keys cannot change.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Role

200 OK → Role
400 401 403 404 422 429 standard errors

DELETE `/roles/{id}` Delete role — `users.manage`

409 ROLE_IN_USE while users have this role; built-in roles cannot be deleted.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/permissions` List permissions — `users.manage`

200 OK → { data: Permission[] }
400 401 403 429 standard errors

GET `/teams` List teams — `users.manage`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: Team[], page: PageInfo }
400 401 403 429 standard errors

POST `/teams` Create team — `users.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Team

201 OK → Team
400 401 403 409 422 429 standard errors

GET `/teams/{id}` Get team — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Team
400 401 403 404 429 standard errors

PATCH `/teams/{id}` Update team — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Team

200 OK → Team
400 401 403 404 409 422 429 standard errors

DELETE `/teams/{id}` Delete team — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/divisions` List divisions — `users.manage`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: Division[], page: PageInfo }
400 401 403 429 standard errors

POST `/divisions` Create division — `users.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Division

201 OK → Division
400 401 403 409 422 429 standard errors

GET `/divisions/{id}` Get division — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Division
400 401 403 404 429 standard errors

PATCH `/divisions/{id}` Update division — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Division

200 OK → Division
400 401 403 404 409 422 429 standard errors

DELETE `/divisions/{id}` Delete division — `users.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/users/{id}/workload` Get user workload — `users.manage`

Used before deactivating a user or switching off lead assignment.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { openLeads: integer, overdue: integer, missedLast30d: integer, avgResponseMinutes: number | null }
400 401 403 404 429 standard errors

Settings

GET `/settings` List settings — `settings.manage`

Keys: leads.response_time, leads.scoring, finance.numbering, …

200 OK → { data: Setting[] }
400 401 403 429 standard errors

PUT `/settings/{key}` Update setting — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`key` *	path	string	

FIELD	TYPE	NOTES
`value` *	object	

200 OK → Setting
400 401 403 404 422 429 standard errors

GET `/lead-stages` List lead stages — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: LeadStage[], page: PageInfo }
400 401 403 429 standard errors

POST `/lead-stages` Create lead stage — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: LeadStage

201 OK → LeadStage
400 401 403 409 422 429 standard errors

GET `/lead-stages/{id}` Get lead stage — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → LeadStage
400 401 403 404 429 standard errors

PATCH `/lead-stages/{id}` Update lead stage — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: LeadStage

200 OK → LeadStage
400 401 403 404 409 422 429 standard errors

DELETE `/lead-stages/{id}` Delete lead stage — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/reasons` List reasons — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`type`	query	`lost` | `invalid` | `reassignment`	

200 OK → { data: Reason[], page: PageInfo }
400 401 403 429 standard errors

POST `/reasons` Create reason — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Reason

201 OK → Reason
400 401 403 409 422 429 standard errors

GET `/reasons/{id}` Get reason — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Reason
400 401 403 404 429 standard errors

PATCH `/reasons/{id}` Update reason — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Reason

200 OK → Reason
400 401 403 404 409 422 429 standard errors

DELETE `/reasons/{id}` Delete reason — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/lead-sources` List lead sources — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: LeadSource[], page: PageInfo }
400 401 403 429 standard errors

POST `/lead-sources` Create lead source — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: LeadSource

201 OK → LeadSource
400 401 403 409 422 429 standard errors

GET `/lead-sources/{id}` Get lead source — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → LeadSource
400 401 403 404 429 standard errors

PATCH `/lead-sources/{id}` Update lead source — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: LeadSource

200 OK → LeadSource
400 401 403 404 409 422 429 standard errors

DELETE `/lead-sources/{id}` Delete lead source — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/currencies` List enabled currencies — `(any signed-in user)`

200 OK → { data: object[] }
400 401 403 429 standard errors

PUT `/currencies` Set enabled currencies and rates — `settings.manage`

FIELD	TYPE	NOTES
`currencies`	object[]	

200 OK → { data: string[] }
400 401 403 429 standard errors

Leads

GET `/leads` List leads — `leads.view`

Only leads in the user's scope, or shared with them, are returned.

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`tab`	query	`all` | `my` | `shared` | `team` | `unassigned` | `deleted` | `new` | `pending` | `scheduled` | `overdue` | `cold`	
`q`	query	string	Name, phone or email
`stageId`	query	string (uuid)	
`ownerUserId`	query	string (uuid)	
`teamId`	query	string (uuid)	
`projectId`	query	string (uuid)	
`sourceId`	query	string (uuid)	
`campaignId`	query	string (uuid)	
`eventId`	query	string (uuid)	
`budgetMin`	query	string	
`budgetMax`	query	string	
`createdFrom`	query	string (date)	
`createdTo`	query	string (date)	
`savedFilterId`	query	string (uuid)	
`sort`	query	string	Default: -score,nextFollowUpAt
`total`	query	boolean	Include total count

200 OK → { data: Lead[], page: PageInfo }
400 401 403 429 standard errors

POST `/leads` Create lead — `leads.create`

409 LEAD_EXISTS (body includes LeadExists) when the phone or email belongs to an existing lead; the screen offers to open it.

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: LeadCreate

201 OK → Lead
400 401 403 409 422 429 standard errors

GET `/leads/check-existing` Check if a lead exists — `leads.create`

Used while typing a new lead to warn: "This lead already exists, owned by ..."

PARAMETER	IN	TYPE	NOTES
`phone`	query	string	
`email`	query	string	

200 OK → { exists: boolean, lead: LeadExists | null }
400 401 403 429 standard errors

GET `/leads/{id}` Get lead — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Lead
400 401 403 404 429 standard errors

PATCH `/leads/{id}` Update lead — `leads.edit`

409 VERSION_CONFLICT if someone else saved first. Shared users need 'edit' access.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: LeadUpdate

200 OK → Lead
400 401 403 404 409 422 429 standard errors

DELETE `/leads/{id}` Delete lead — `leads.delete`

Soft delete; appears in the Deleted tab.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

POST `/leads/{id}/restore` Restore lead — `leads.delete`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Lead
400 401 403 404 429 standard errors

POST `/leads/{id}/stage` Change lead stage — `leads.edit`

422 REASON_REQUIRED for Lost/Invalid. Logged in stage history and the timeline.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: StageChange

200 OK → Lead
400 401 403 404 422 429 standard errors

POST `/leads/{id}/assign` Assign lead — `leads.assign`

422 REASON_REQUIRED when reassigning; 422 USER_NOT_RECEIVING_LEADS if assignment is off for the target user. Starts the 10-minute response timer.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Assign

200 OK → Lead
400 401 403 404 422 429 standard errors

GET `/leads/{id}/timeline` Get lead timeline — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`types`	query	string	Comma-separated activity types

200 OK → { data: Activity[], page: PageInfo }
400 401 403 404 429 standard errors

GET `/leads/{id}/inquiries` List lead enquiries — `leads.view`

All enquiries including re-enquiries, with original form answers and source IDs.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Inquiry[] }
400 401 403 404 429 standard errors

POST `/leads/{id}/notes` Add note — `leads.edit`

Counts as the owner's response for the 10-minute rule.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Note

201 OK → Activity
400 401 403 404 429 standard errors

GET `/leads/{id}/shares` List lead shares — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: LeadShare[] }
400 401 403 404 429 standard errors

POST `/leads/{id}/shares` Share lead — `leads.share`

Only the owner or a manager above them. Shared user is notified.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Share

201 OK → LeadShare
400 401 403 404 422 429 standard errors

DELETE `/leads/{id}/shares/{userId}` Remove lead share — `leads.share`

PARAMETER	IN	TYPE	NOTES
`userId` *	path	string (uuid)	
`id` *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

PUT `/leads/{id}/projects` Set lead projects — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`projectIds` *	string (uuid)[]	
`primaryProjectId`	string (uuid)	

200 OK → Lead
400 401 403 404 429 standard errors

GET `/leads/{id}/attachments` List lead attachments — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Attachment[] }
400 401 403 404 429 standard errors

POST `/leads/{id}/attachments` Attach file to lead — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`category` *	string	

201 OK → Attachment
400 401 403 404 429 standard errors

POST `/leads/bulk` Bulk update leads — `leads.assign`

Permission depends on action: assign → leads.assign, stage → leads.edit, delete/restore → leads.delete.

Body: BulkLeadAction

200 OK → BulkResult
400 401 403 429 standard errors

POST `/leads/imports` Start lead import — `leads.import`

Upload the Excel/CSV via /files/uploads first. Returns a preview; call confirm to import.

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`sourceId` *	string (uuid)	
`mapping`	object	

201 OK → LeadImport
400 401 403 429 standard errors

GET `/leads/imports/{id}` Get lead import — `leads.import`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → LeadImport
400 401 403 404 429 standard errors

POST `/leads/imports/{id}/confirm` Confirm lead import — `leads.import`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

202 Import started → LeadImport
400 401 403 404 429 standard errors

GET `/saved-filters` List saved filters — `(any signed-in user)`

PARAMETER	IN	TYPE	NOTES
`entity`	query	string	

200 OK → { data: SavedFilter[] }
400 401 403 429 standard errors

POST `/saved-filters` Create saved filter — `(any signed-in user)`

Body: SavedFilter

201 OK → SavedFilter
400 401 403 429 standard errors

DELETE `/saved-filters/{id}` Delete saved filter — `(owner)`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

POST `/leads/{id}/calls` Log call — `leads.edit`

Adds the call to the timeline and sets the next follow-up. Counts as the owner's response for the 10-minute rule. wrong_number prompts moving the lead to Invalid.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: CallLog

201 OK → Activity
400 401 403 404 422 429 standard errors

POST `/leads/{id}/priority` Set lead priority — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`priority` *	`low` | `medium` | `high`	

200 OK → Lead
400 401 403 404 429 standard errors

Assignment

GET `/assignment-rules` List assignment rules — `settings.manage`

200 OK → { data: AssignmentRule[] }
400 401 403 429 standard errors

POST `/assignment-rules` Create assignment rule — `settings.manage`

Body: AssignmentRule

201 OK → AssignmentRule
400 401 403 429 standard errors

PATCH `/assignment-rules/{id}` Update assignment rule — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: AssignmentRule

200 OK → AssignmentRule
400 401 403 404 429 standard errors

DELETE `/assignment-rules/{id}` Delete assignment rule — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

PUT `/assignment-rules/order` Reorder assignment rules — `settings.manage`

FIELD	TYPE	NOTES
`ruleIds` *	string (uuid)[]	

204 Done
400 401 403 429 standard errors

Marketing

GET `/campaigns` List campaigns — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`sourceId`	query	string (uuid)	
`status`	query	string	

200 OK → { data: Campaign[], page: PageInfo }
400 401 403 429 standard errors

POST `/campaigns` Create campaign — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Campaign

201 OK → Campaign
400 401 403 409 422 429 standard errors

GET `/campaigns/{id}` Get campaign — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Campaign
400 401 403 404 429 standard errors

PATCH `/campaigns/{id}` Update campaign — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Campaign

200 OK → Campaign
400 401 403 404 409 422 429 standard errors

GET `/campaigns/{id}/spend` List campaign spend — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`from`	query	string (date)	
`to`	query	string (date)	

200 OK → { data: AdSpend[] }
400 401 403 404 429 standard errors

PUT `/campaigns/{id}/spend` Enter campaign spend — `settings.manage`

Manual entry; API-imported days are overwritten only by the import.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`days`	AdSpend[]	

204 Done
400 401 403 404 429 standard errors

GET `/events` List events — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: Event[], page: PageInfo }
400 401 403 429 standard errors

POST `/events` Create event — `leads.create`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Event

201 OK → Event
400 401 403 409 422 429 standard errors

GET `/events/{id}` Get event — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Event
400 401 403 404 429 standard errors

PATCH `/events/{id}` Update event — `leads.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Event

200 OK → Event
400 401 403 404 409 422 429 standard errors

DELETE `/events/{id}` Delete event — `leads.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

Tasks

GET `/tasks` List tasks — `(own tasks; managers within scope)`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`assignedUserId`	query	string (uuid)	
`leadId`	query	string (uuid)	
`status`	query	string	
`type`	query	string	
`from`	query	string (date)	
`to`	query	string (date)	
`view`	query	`day` | `week` | `month` | `list`	
`overdue`	query	boolean	

200 OK → { data: Task[], page: PageInfo }
400 401 403 429 standard errors

POST `/tasks` Create task — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Task

201 OK → Task
400 401 403 429 standard errors

PATCH `/tasks/{id}` Update task — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Task

200 OK → Task
400 401 403 404 429 standard errors

POST `/tasks/{id}/complete` Complete task — `leads.edit`

422 OUTCOME_REQUIRED / NEXT_ACTION_REQUIRED. Updates the lead's next follow-up and timeline; counts as a response for the 10-minute rule.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: TaskComplete

200 OK → { task: Task, nextTask: Task | null }
400 401 403 404 422 429 standard errors

POST `/tasks/{id}/cancel` Cancel task — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Task
400 401 403 404 429 standard errors

Meetings

GET /meetings List meetings

leads.view

PARAMETER	IN	TYPE	NOTES
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
organizerUserId	query	string (uuid)	
leadId	query	string (uuid)	
from	query	string (date)	
to	query	string (date)	
status	query	string	

200 OK → { data: Meeting[], page: PageInfo }
400 401 403 429 standard errors

POST /meetings Create meeting

leads.edit

Creates the Google Meet / Teams link, the calendar event and the client confirmation. 409 AGENT_BUSY on overlap.

PARAMETER	IN	TYPE	NOTES
Idempotency-Key	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: MeetingCreate

201 OK → Meeting
400 401 403 409 422 429 standard errors

GET /meetings/{id} Get meeting

leads.view

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → Meeting
400 401 403 404 429 standard errors

PATCH /meetings/{id} Reschedule meeting

leads.edit

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
startsAt	string (date-time)	
endsAt	string (date-time)	
location	string	

200 OK → Meeting
400 401 403 404 409 429 standard errors

POST `/meetings/{id}/cancel` Cancel meeting — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: CancelRequest

200 OK → Meeting
400 401 403 404 429 standard errors

POST `/meetings/{id}/complete` Complete meeting — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`status` *	`completed` | `no_show`	
`note`	string	
`nextAction`	object	

200 OK → Meeting
400 401 403 404 429 standard errors

GET `/availability` Get agent free slots — `leads.view`

PARAMETER	IN	TYPE	NOTES
`userId` *	query	string (uuid)	
`from`	query	string (date)	
`to`	query	string (date)	
`durationMinutes`	query	integer	

200 OK → { slots: object[] }
400 401 403 429 standard errors

GET `/booking-pages/me` Get my booking page — `(any signed-in user)`

200 OK → BookingPage
400 401 403 429 standard errors

PUT `/booking-pages/me` Save my booking page — `(any signed-in user)`

409 SLUG_TAKEN if the link name is used.

Body: BookingPage

200 OK → BookingPage
400 401 403 409 429 standard errors

Public booking

GET `/public/booking/{slug}` Get public booking page — `public`

No login. Shows the agent's free slots in the client's time zone.

PARAMETER	IN	TYPE	NOTES
`slug` *	path	string	
`from`	query	string (date)	
`to`	query	string (date)	
`timezone`	query	string	

200 OK → PublicBookingPage
400 404 429 standard errors

POST `/public/booking/{slug}` Book a slot — `public`

No login; protected by rate limiting and a bot check (Cloudflare Turnstile token in header CF-Turnstile-Token). Matches the client to an existing lead by phone/email (re-enquiry) or creates one owned by the page's agent. 409 SLOT_TAKEN.

PARAMETER	IN	TYPE	NOTES
`slug` *	path	string	

Body: PublicBookingRequest

201 OK → PublicBookingConfirmation
400 404 409 429 standard errors

Messaging

GET `/conversations` List conversations — `messages.send`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`channel`	query	string	
`unread`	query	boolean	
`leadId`	query	string (uuid)	

200 OK → { data: Conversation[], page: PageInfo }
400 401 403 429 standard errors

GET `/conversations/{id}/messages` List messages — `messages.send`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: Message[], page: PageInfo }
400 401 403 404 429 standard errors

POST `/conversations/{id}/read` Mark conversation read — `messages.send`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

POST `/leads/{id}/messages` Send message to lead — `messages.send`

422 OPTED_OUT, TEMPLATE_REQUIRED (WhatsApp outside the 24-hour window).

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: MessageSend

202 Queued for sending → Message
400 401 403 404 422 429 standard errors

GET `/leads/{id}/consents` Get lead consent history — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Consent[] }
400 401 403 404 429 standard errors

POST `/leads/{id}/consents` Record consent — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Consent

201 OK → Consent
400 401 403 404 429 standard errors

GET `/message-templates` List message templates — `messages.send`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`channel`	query	string	
`status`	query	string	

200 OK → { data: MessageTemplate[], page: PageInfo }
400 401 403 429 standard errors

POST `/message-templates` Create message template — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: MessageTemplate

201 OK → MessageTemplate
400 401 403 409 422 429 standard errors

GET `/message-templates/{id}` Get message template — `messages.send`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → MessageTemplate
400 401 403 404 429 standard errors

PATCH `/message-templates/{id}` Update message template — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: MessageTemplate

200 OK → MessageTemplate
400 401 403 404 409 422 429 standard errors

DELETE `/message-templates/{id}` Delete message template — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

POST `/message-templates/{id}/submit` Submit template to WhatsApp

`settings.manage`

Sends a WhatsApp template to Meta for approval; status updates by webhook.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → MessageTemplate
400 401 403 404 429 standard errors

GET `/message-campaigns` List bulk campaigns

`campaigns.create`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`status`	query	string	

200 OK → { data: MessageCampaign[], page: PageInfo }
400 401 403 429 standard errors

POST `/message-campaigns` Create bulk campaign

`campaigns.create`

Body: MessageCampaign

201 OK → MessageCampaign
400 401 403 429 standard errors

GET `/message-campaigns/{id}` Get bulk campaign

`campaigns.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → MessageCampaign
400 401 403 404 429 standard errors

PATCH `/message-campaigns/{id}` Update bulk campaign

`campaigns.create`

Only while draft or rejected.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: MessageCampaign

200 OK → MessageCampaign
400 401 403 404 409 429 standard errors

POST `/message-campaigns/{id}/preview-audience` Preview campaign audience — `campaigns.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { count: integer, excludedOptedOut: integer, sample: IdRef[] }
400 401 403 404 429 standard errors

POST `/message-campaigns/{id}/test` Send campaign test — `campaigns.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`to` *	string	

202 OK
400 401 403 404 429 standard errors

POST `/message-campaigns/{id}/submit` Submit campaign for approval — `campaigns.create`

422 TEST_REQUIRED if no test was sent.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → MessageCampaign
400 401 403 404 422 429 standard errors

POST `/message-campaigns/{id}/approve` Approve campaign — `campaigns.approve`

422 SELF_APPROVAL: the creator cannot approve their own campaign.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → MessageCampaign
400 401 403 404 422 429 standard errors

POST `/message-campaigns/{id}/reject` Reject campaign — `campaigns.approve`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`note` *	string	

200 OK → MessageCampaign
400 401 403 404 429 standard errors

POST `/message-campaigns/{id}/cancel` Cancel campaign — `campaigns.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → MessageCampaign
400 401 403 404 429 standard errors

GET `/message-campaigns/{id}/report` Get campaign report — `campaigns.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → CampaignReport
400 401 403 404 429 standard errors

Projects

GET `/developers` List developers — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: Developer[], page: PageInfo }
400 401 403 429 standard errors

POST `/developers` Create developer — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Developer

201 OK → Developer
400 401 403 409 422 429 standard errors

GET `/developers/{id}` Get developer — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Developer
400 401 403 404 429 standard errors

PATCH `/developers/{id}` Update developer — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Developer

200 OK → Developer
400 401 403 404 409 422 429 standard errors

DELETE `/developers/{id}` Delete developer — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/projects` List projects — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`developerId`	query	string (uuid)	
`q`	query	string	
`isActive`	query	boolean	

200 OK → { data: Project[], page: PageInfo }
400 401 403 429 standard errors

POST `/projects` Create project — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Project

201 OK → Project
400 401 403 409 422 429 standard errors

GET `/projects/{id}` Get project — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Project
400 401 403 404 429 standard errors

PATCH `/projects/{id}` Update project — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Project

200 OK → Project
400 401 403 404 409 422 429 standard errors

DELETE `/projects/{id}` Delete project — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/projects/{id}/buildings` List buildings — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Building[] }
400 401 403 404 429 standard errors

POST `/projects/{id}/buildings` Create building — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Building

201 OK → Building
400 401 403 404 429 standard errors

PATCH `/buildings/{id}` Update building — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Building

200 OK → Building
400 401 403 404 429 standard errors

GET `/units` List units

`leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`projectId`	query	string (uuid)	
`buildingId`	query	string (uuid)	
`propertyType`	query	string	
`bedrooms`	query	integer	
`priceMin`	query	string	
`priceMax`	query	string	
`status`	query	string	

200 OK → { data: Unit[], page: PageInfo }
400 401 403 429 standard errors

POST `/units` Create unit

`projects.manage`

Body: Unit

201 OK → Unit
400 401 403 409 429 standard errors

GET `/units/{id}` Get unit

`leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Unit
400 401 403 404 429 standard errors

PATCH `/units/{id}` Update unit details

`projects.manage`

Price and status are changed through /units/{id}/status so the history is kept.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Unit

200 OK → Unit
400 401 403 404 429 standard errors

POST `/units/{id}/status` Change unit status or price — `units.change_status`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: UnitStatusChange

200 OK → Unit
400 401 403 404 409 422 429 standard errors

GET `/units/{id}/history` Get unit price and status history — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: UnitChange[] }
400 401 403 404 429 standard errors

POST `/projects/{id}/attachments` Attach file to project — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`category` *	string	
`sortOrder`	integer	

201 OK → Attachment
400 401 403 404 429 standard errors

GET `/units/{id}/attachments` List unit attachments — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Attachment[] }
400 401 403 404 429 standard errors

POST `/units/{id}/attachments` Attach file to unit — `projects.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`category` *	string	

201 OK → Attachment
400 401 403 404 429 standard errors

Deals

GET `/deals` List deals — `leads.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`leadId`	query	string (uuid)	
`projectId`	query	string (uuid)	
`stage`	query	string	
`ownerUserId`	query	string (uuid)	

200 OK → { data: Deal[], page: PageInfo }
400 401 403 429 standard errors

POST `/deals` Create deal — `leads.edit`

Body: Deal

201 OK → Deal
400 401 403 422 429 standard errors

GET `/deals/{id}` Get deal — `leads.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Deal
400 401 403 404 429 standard errors

PATCH `/deals/{id}` Update deal — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Deal

200 OK → Deal
400 401 403 404 429 standard errors

POST `/deals/{id}/stage` Change deal stage — `leads.edit`

Moves the unit to On Hold / Booked / Sold and updates the lead timeline.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: DealStageChange

200 OK → Deal
400 401 403 404 409 422 429 standard errors

GET `/deals/{id}/attachments` List deal attachments — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Attachment[] }
400 401 403 404 429 standard errors

POST `/deals/{id}/attachments` Attach file to deal — `leads.edit`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`category` *	string	

201 OK → Attachment
400 401 403 404 429 standard errors

Finance

GET `/counterparties` List counterparties — `finance.create`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`type`	query	string	
`q`	query	string	

200 OK → { data: Counterparty[], page: PageInfo }
400 401 403 429 standard errors

POST `/counterparties` Create counterparty — `finance.create`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Counterparty

201 OK → Counterparty
400 401 403 409 422 429 standard errors

GET `/counterparties/{id}` Get counterparty — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Counterparty
400 401 403 404 429 standard errors

PATCH `/counterparties/{id}` Update counterparty — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: Counterparty

200 OK → Counterparty
400 401 403 404 409 422 429 standard errors

GET `/finance-documents` List financial documents — `finance.create`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`docType`	query	string	
`status`	query	string	
`counterpartyId`	query	string (uuid)	
`leadId`	query	string (uuid)	
`dealId`	query	string (uuid)	
`from`	query	string (date)	
`to`	query	string (date)	

200 OK → { data: FinanceDocument[], page: PageInfo }
400 401 403 429 standard errors

POST `/finance-documents` Create financial document — `finance.create`

Created as draft; totals are calculated by the server.

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: FinanceDocument

201 OK → FinanceDocument
400 401 403 429 standard errors

GET `/finance-documents/{id}` Get financial document — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → FinanceDocument
400 401 403 404 429 standard errors

PATCH `/finance-documents/{id}` Update draft document — `finance.create`

422 NOT_DRAFT: approved documents cannot be edited; cancel and reissue.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: FinanceDocument

200 OK → FinanceDocument
400 401 403 404 409 422 429 standard errors

POST `/finance-documents/{id}/approve` Approve document — `finance.approve`

Assigns the number (e.g. INV-2026-0042), generates the PDF and locks amounts.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → FinanceDocument
400 401 403 404 422 429 standard errors

POST `/finance-documents/{id}/send` Send document — `finance.approve`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`to`	string[]	
`message`	string	

200 OK → FinanceDocument
400 401 403 404 429 standard errors

POST `/finance-documents/{id}/cancel` Cancel document — `finance.approve`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: CancelRequest

200 OK → FinanceDocument
400 401 403 404 429 standard errors

GET `/finance-documents/{id}/pdf` Download document PDF — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → DownloadLink
400 401 403 404 429 standard errors

GET `/finance-documents/{id}/history` Get document history — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: DocumentEvent[] }
400 401 403 404 429 standard errors

GET `/finance-documents/{id}/payments` List payments — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Payment[] }
400 401 403 404 429 standard errors

POST `/finance-documents/{id}/payments` Record payment or refund — `finance.approve`

422 AMOUNT_EXCEEDS_BALANCE / CURRENCY_MISMATCH. Status becomes Partially Paid, Paid or Refunded automatically.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Payment

201 OK → FinanceDocument
400 401 403 404 422 429 standard errors

GET `/commissions` List commissions — `finance.create`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`userId`	query	string (uuid)	
`dealId`	query	string (uuid)	
`status`	query	string	

200 OK → { data: Commission[], page: PageInfo }
400 401 403 429 standard errors

POST `/commissions` Create commission — `finance.create`

Body: Commission

201 OK → Commission
400 401 403 409 429 standard errors

POST `/commissions/{id}/approve` Approve commission — `finance.approve`

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → Commission
400 401 403 404 429 standard errors

POST `/commissions/{id}/mark-paid` Mark commission paid — `finance.approve`

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
paidOn *	string (date)	

200 OK → Commission
400 401 403 404 429 standard errors

GET `/finance-documents/{id}/attachments` List finance document attachments — `finance.create`

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → { data: Attachment[] }
400 401 403 404 429 standard errors

POST `/finance-documents/{id}/attachments` Attach file to finance document — `finance.create`

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
fileId *	string (uuid)	
category *	string	

201 OK → Attachment
400 401 403 404 429 standard errors

`GET` `/commissions/{id}/attachments` List commission attachments — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { data: Attachment[] }
400 401 403 404 429 standard errors

`POST` `/commissions/{id}/attachments` Attach file to commission — `finance.create`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`category` *	string	

201 OK → Attachment
400 401 403 404 429 standard errors

Attendance

`POST` `/attendance/check-in` Check in — `(own)`

409 ALREADY_CHECKED_IN; 422 LOCATION_REQUIRED / OUTSIDE_OFFICE_RADIUS. Status (present / late) is set by policy.

Body: CheckIn

201 OK → AttendanceRecord
400 401 403 409 422 429 standard errors

`POST` `/attendance/check-out` Check out — `(own)`

Body: CheckIn

200 OK → AttendanceRecord
400 401 403 409 429 standard errors

`GET` `/attendance/records` List attendance records — `attendance.view`

PARAMETER	IN	TYPE	NOTES
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
userId	query	string (uuid)	
teamId	query	string (uuid)	
from	query	string (date)	
to	query	string (date)	
status	query	string	

200 OK → { data: AttendanceRecord[], page: PageInfo }
400 401 403 429 standard errors

`POST` `/attendance/records/{id}/corrections` Request attendance correction — `(own)`

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

Body: AttendanceCorrection

201 OK → AttendanceCorrection
400 401 403 404 429 standard errors

`GET` `/attendance/corrections` List correction requests — `attendance.approve`

PARAMETER	IN	TYPE	NOTES
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
status	query	string	

200 OK → { data: AttendanceCorrection[], page: PageInfo }
400 401 403 429 standard errors

`POST` `/attendance/corrections/{id}/review` Review correction — `attendance.approve`

422 SELF_REVIEW. On approval the record is updated and the original values are kept.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

Body: Review

200 OK → AttendanceCorrection
400 401 403 404 422 429 standard errors

GET `/leave-requests` List leave requests — `attendance.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`userId`	query	string (uuid)	
`status`	query	string	
`from`	query	string (date)	
`to`	query	string (date)	

200 OK → { data: LeaveRequest[], page: PageInfo }
400 401 403 429 standard errors

POST `/leave-requests` Request leave — `(own)`

409 LEAVE_OVERLAP.

Body: LeaveRequest

201 OK → LeaveRequest
400 401 403 409 429 standard errors

POST `/leave-requests/{id}/review` Review leave request — `attendance.approve`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Review

200 OK → LeaveRequest
400 401 403 404 409 422 429 standard errors

POST `/leave-requests/{id}/cancel` Cancel leave request — `(own)`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → LeaveRequest
400 401 403 404 429 standard errors

GET `/attendance-policies` List attendance policies — `attendance.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: AttendancePolicy[], page: PageInfo }
400 401 403 429 standard errors

POST `/attendance-policies` Create attendance policy — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: AttendancePolicy

201 OK → AttendancePolicy
400 401 403 409 422 429 standard errors

GET `/attendance-policies/{id}` Get attendance policy — `attendance.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → AttendancePolicy
400 401 403 404 429 standard errors

PATCH `/attendance-policies/{id}` Update attendance policy — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: AttendancePolicy

200 OK → AttendancePolicy
400 401 403 404 409 422 429 standard errors

DELETE `/attendance-policies/{id}` Delete attendance policy — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/holidays` List holidays — `attendance.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: Holiday[], page: PageInfo }
400 401 403 429 standard errors

POST `/holidays` Create holiday — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: Holiday

201 OK → Holiday
400 401 403 409 422 429 standard errors

DELETE `/holidays/{id}` Delete holiday — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

GET `/leave-types` List leave types — `attendance.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: LeaveType[], page: PageInfo }
400 401 403 429 standard errors

POST `/leave-types` Create leave type — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`Idempotency-Key`	header	string	Optional. Retrying a POST with the same key returns the first result instead of creating a duplicate.

Body: LeaveType

201 OK → LeaveType
400 401 403 409 422 429 standard errors

PATCH `/leave-types/{id}` Update leave type — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`If-Match`	header	string	Optional ETag (version) for optimistic locking

Body: LeaveType

200 OK → LeaveType
400 401 403 404 409 422 429 standard errors

DELETE `/leave-types/{id}` Delete leave type — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 409 429 standard errors

Dashboard

GET `/dashboard` Get dashboard — `leads.view`

Data is limited to the user's scope. Figures are at most 5 minutes old.

PARAMETER	IN	TYPE	NOTES
`from`	query	string (date)	
`to`	query	string (date)	
`projectId`	query	string (uuid)	
`sourceId`	query	string (uuid)	
`campaignId`	query	string (uuid)	
`teamId`	query	string (uuid)	
`userId`	query	string (uuid)	

200 OK → Dashboard
400 401 403 429 standard errors

Reports

GET `/reports` List reports — `leads.view`

Only reports the user may view (finance and attendance reports need their module permissions).

200 OK → { data: ReportDefinition[] }
400 401 403 429 standard errors

`GET` `/reports/{key}` Run report — `leads.view`

PARAMETER	IN	TYPE	NOTES
key *	path	string	
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
from	query	string (date)	
to	query	string (date)	
projectId	query	string (uuid)	
sourceId	query	string (uuid)	
campaignId	query	string (uuid)	
teamId	query	string (uuid)	
userId	query	string (uuid)	

200 OK → ReportResult
400 401 403 404 429 standard errors

`POST` `/reports/{key}/exports` Export report — `reports.export`

Admin only by default (leads.export / reports.export / attendance.export by report). Totals equal the on-screen totals. Every export is logged. Large exports run in the background; the user is notified.

PARAMETER	IN	TYPE	NOTES
key *	path	string	

Body: ExportRequest

202 OK → Export
400 401 403 404 429 standard errors

`GET` `/exports/{id}` Get export — `reports.export`

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → Export
400 401 403 404 429 standard errors

POST `/integrations/{id}/reconciliations` Run Meta reconciliation — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`periodStart` *	string (date-time)	
`periodEnd` *	string (date-time)	
`recoverMissing`	boolean	

202 OK → Reconciliation
400 401 403 404 429 standard errors

GET `/reconciliations` List reconciliation runs — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`integrationId`	query	string (uuid)	

200 OK → { data: Reconciliation[], page: PageInfo }
400 401 403 429 standard errors

GET `/exports` List export log — `audit.view`

Every export with user, report, filters, format, rows and time.

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`userId`	query	string (uuid)	
`from`	query	string (date)	
`to`	query	string (date)	

200 OK → { data: Export[], page: PageInfo }
400 401 403 429 standard errors

Integrations

GET `/integrations` List integrations — `settings.manage`

Includes the user's own calendar/mail connections for non-admins.

200 OK → { data: Integration[] }
400 401 403 429 standard errors

POST `/integrations` Connect integration — `settings.manage`

OAuth providers return authorizeUrl; user-scope providers (Google, Microsoft, Gmail) only need the user to be signed in.

Body: IntegrationConnect

201 OK → ConnectStart
400 401 403 429 standard errors

GET `/integrations/{id}` Get integration — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → Integration
400 401 403 404 429 standard errors

PATCH `/integrations/{id}` Update integration settings — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

Body: Integration

200 OK → Integration
400 401 403 404 429 standard errors

DELETE `/integrations/{id}` Disconnect integration — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

POST `/integrations/{id}/test` Test integration — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { ok: boolean, detail: string }
400 401 403 404 429 standard errors

POST `/integrations/{id}/retry` Retry failed events — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

202 OK → { requeued: integer }
400 401 403 404 429 standard errors

GET `/integrations/{id}/sync-runs` List sync runs

settings.manage

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	

200 OK → { data: SyncRun[], page: PageInfo }
400 401 403 404 429 standard errors

GET `/oauth/{provider}/callback` OAuth callback

public

Provider redirects here after the user approves access; state is a signed, single-use value.

PARAMETER	IN	TYPE	NOTES
provider *	path	string	
code	query	string	
state *	query	string	

302 Redirects back to the Integrations screen
400 404 429 standard errors

POST `/webhook-sources` Create general webhook

settings.manage

Creates a secure inbound URL for any system: website, Google Sheets (Apps Script), Excel (Power Automate), Make.com. The secret is shown once.

Body: WebhookSource

201 OK → WebhookSource
400 401 403 429 standard errors

PUT `/webhook-sources/{id}/mapping` Update webhook field mapping

settings.manage

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
fieldMapping *	object	

200 OK → WebhookSource
400 401 403 404 429 standard errors

POST `/webhook-sources/{id}/rotate-secret` Rotate webhook secret

settings.manage

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → WebhookSource
400 401 403 404 429 standard errors

GET `/integrations/{id}/events` List incoming events — `settings.manage`

Failures with error detail, for the Integrations screen.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`status`	query	string	

200 OK → { data: WebhookEvent[], page: PageInfo }
400 401 403 404 429 standard errors

POST `/webhook-events/{id}/retry` Retry one incoming event — `settings.manage`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

202 OK → WebhookEvent
400 401 403 404 429 standard errors

Files

POST `/files/uploads` Request upload — `(any signed-in user)`

Returns a 10-minute S3 upload link. 422 FILE_TYPE_NOT_ALLOWED / FILE_TOO_LARGE (25 MB).

Body: UploadRequest

201 OK → UploadTicket
400 401 403 422 429 standard errors

POST `/files/{id}/complete` Confirm upload — `(uploader)`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → { fileId: string (uuid), sizeBytes: integer }
400 401 403 404 429 standard errors

GET `/files/{id}/download` Get download link — `(record access)`

5-minute link; only for files the user may see.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → DownloadLink
400 401 403 404 429 standard errors

DELETE `/attachments/{id}` Remove attachment (edit access to the record)

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

Notifications

GET `/notifications` List notifications (own)

PARAMETER	IN	TYPE	NOTES
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
unread	query	boolean	

200 OK → { data: Notification[], page: PageInfo }
400 401 403 429 standard errors

POST `/notifications/{id}/read` Mark notification read (own)

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

204 Done
400 401 403 404 429 standard errors

POST `/notifications/read-all` Mark all notifications read (own)

204 Done
400 401 403 429 standard errors

POST `/push-devices` Register push device (own)

Body: PushDevice

204 Done
400 401 403 429 standard errors

DELETE `/push-devices/{token}` Remove push device (own)

PARAMETER	IN	TYPE	NOTES
token *	path	string	

204 Done
400 401 403 404 429 standard errors

Audit

GET `/audit-logs` List audit log — `audit.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`entityType`	query	string	
`entityId`	query	string (uuid)	
`userId`	query	string (uuid)	
`action`	query	string	
`from`	query	string (date)	
`to`	query	string (date)	

200 OK → { data: AuditEntry[], page: PageInfo }
400 401 403 429 standard errors

Public sign-up

GET `/public/plans` List plans for pricing page — `public`

Active plans with prices per user (excluding VAT). No login.

200 OK → { data: Plan[] }
400 429 standard errors

POST `/public/signup` Start sign-up — `public`

Step 1. Checks the bot token, plan and RERA / licence fields, and emails a 6-digit code. Returns a Stripe card setup, because a card is required to start the 14-day trial. 409 EMAIL_ALREADY_REGISTERED; 422 LICENCE_REQUIRED, PLAN_NOT_SELF_SERVICE (Enterprise goes through sales).

Body: SignupStart

201 OK → Signup
400 409 422 429 standard errors

POST `/public/signup/{id}/verify-email` Verify sign-up email — `public`

Step 2. 422 CODE_INVALID or CODE_EXPIRED (5 tries, 15 minutes).

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`code` *	string	

200 OK → Signup
400 404 422 429 standard errors

POST `/public/signup/{id}/resend-code` Resend sign-up code — public

At most once a minute.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

204 Done
400 404 429 standard errors

POST `/public/signup/{id}/complete` Complete sign-up and start trial — public

Step 3, after the card is saved with Stripe. Creates the company with default roles, stages and settings, the Admin user and a 14-day trial subscription; licence verification by our team starts in the background. 422 CARD_REQUIRED, EMAIL_NOT_VERIFIED.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`setupIntentId` *	string	

201 OK → SignupComplete
400 404 422 429 standard errors

POST `/public/sales-enquiries` Contact sales — public

Enterprise and 'talk to sales' form. Protected by the bot check.

Body: SalesEnquiry

201 OK → { received: boolean }
400 429 standard errors

Subscription & billing

GET `/billing/subscription` Get subscription — billing.view

Plan, seats used / paid, status, trial or renewal date and the next invoice estimate.

200 OK → Subscription
400 401 403 429 standard errors

POST `/billing/subscription/change-plan` Change plan or billing cycle — `billing.manage`

Upgrades and a switch to yearly apply now with a pro-rata charge; downgrades apply at the next renewal. 422 PLAN_NOT_ALLOWED (e.g. Solo for a brokerage).

FIELD	TYPE	NOTES
`planId` *	string (uuid)	
`billingCycle`	`monthly` | `yearly`	

200 OK → Subscription
400 401 403 422 429 standard errors

PUT `/billing/subscription/seats` Change number of seats — `billing.manage`

More seats: charged pro-rata now. Fewer seats: from the next renewal. 422 SEATS_BELOW_ACTIVE_USERS (deactivate users first).

FIELD	TYPE	NOTES
`seats` *	integer	

200 OK → Subscription
400 401 403 422 429 standard errors

PUT `/billing/subscription/payment-method` Change payment method — `billing.manage`

Bank transfer: invoices are emailed with our bank details and paid by transfer.

FIELD	TYPE	NOTES
`method` *	`card` | `bank_transfer`	

200 OK → Subscription
400 401 403 429 standard errors

POST `/billing/card-setup` Start card update — `billing.manage`

Returns a Stripe card setup for adding or replacing the card.

201 OK → CardSetup
400 401 403 429 standard errors

POST `/billing/subscription/cancel` Cancel subscription — `billing.manage`

Cancels at the end of the paid period; no refund for the started period. Data stays available for export for 90 days after the end, then is deleted.

FIELD	TYPE	NOTES
`reason`	string	

200 OK → Subscription
400 401 403 429 standard errors

POST `/billing/subscription/resume` Undo cancellation — `billing.manage`

Only before the period ends.

200 OK → Subscription
400 401 403 422 429 standard errors

GET `/billing/invoices` List subscription invoices — `billing.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`status`	query	`open` | `paid` | `void` | `uncollectible`	

200 OK → { data: BillingInvoice[], page: PageInfo }
400 401 403 429 standard errors

GET `/billing/invoices/{id}` Get subscription invoice — `billing.view`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → BillingInvoice
400 401 403 404 429 standard errors

GET `/billing/invoices/{id}/pdf` Download invoice PDF — `billing.view`

UAE tax invoice with our TRN and 5% VAT.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

200 OK → DownloadLink
400 401 403 404 429 standard errors

POST `/billing/invoices/{id}/bank-transfers` Submit bank transfer details — `billing.manage`

Records the transfer as pending; our team confirms it when the money arrives.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`amount` *	string	Decimal as string, 2 places
`bankReference` *	string	
`proofFileId`	string (uuid)	

201 OK → BillingPayment
400 401 403 404 429 standard errors

GET `/billing/payments` List subscription payments — `billing.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: BillingPayment[], page: PageInfo }
400 401 403 429 standard errors

GET `/billing/usage` Get usage — `billing.view`

200 OK → BillingUsage
400 401 403 429 standard errors

GET `/billing/history` List subscription history — `billing.view`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: SubscriptionEvent[], page: PageInfo }
400 401 403 429 standard errors

POST `/billing/data-export` Export all company data — `billing.manage`

Full export (Excel/CSV zip) of all company data; also available while read-only and for 90 days after cancellation.

202 OK → Export
400 401 403 429 standard errors

Owner console

GET `/platform/me` Get platform admin — `(platform: any)`

Owner console uses its own login (separate Cognito user pool, two-factor required) and its own database role.

200 OK → PlatformAdmin
400 401 403 429 standard errors

GET `/platform/metrics` Get business metrics — `(platform: owner, support)`

Monthly recurring revenue, paying and trial companies, paid seats, conversion and churn.

200 OK → PlatformMetrics
400 401 403 429 standard errors

GET /platform/tenants List companies

(platform: owner, support)

PARAMETER	IN	TYPE	NOTES
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
q	query	string	Name, email, licence
subscriptionStatus	query	trialing | active | past_due | read_only | canceled	
verificationStatus	query	pending | verified | rejected	
planKey	query	string	

200 OK → { data: PlatformTenant[], page: PageInfo }

400 401 403 429 standard errors

GET /platform/tenants/{id} Get company

(platform: owner, support)

Profile, subscription, usage and verification. Does not show the company's leads or messages.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → PlatformTenant

400 401 403 404 429 standard errors

POST /platform/tenants/{id}/verify Verify company licence

(platform: owner, support)

Checks trade licence / RERA ORN or broker card BRN. WhatsApp can be connected only after verification.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
result *	verified | rejected	
note	string	

200 OK → PlatformTenant

400 401 403 404 429 standard errors

POST `/platform/tenants/{id}/suspend` Suspend company (platform: owner)

Blocks login for all users of the company (e.g. abuse). Data is kept.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
reason *	string	

200 OK → PlatformTenant
400 401 403 404 429 standard errors

POST `/platform/tenants/{id}/unsuspend` Unsuspend company (platform: owner)

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

200 OK → PlatformTenant
400 401 403 404 429 standard errors

POST `/platform/tenants/{id}/extend` Extend trial or grace period (platform: owner)

Free extra days, e.g. for a pilot brokerage. Logged in the subscription history.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
days *	integer	
note *	string	

200 OK → PlatformTenant
400 401 403 404 429 standard errors

PATCH `/platform/subscriptions/{id}` Set custom price or seats (platform: owner)

Enterprise deals: custom price per user.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
pricePerSeat	string	Decimal as string, 2 places
seats	integer	
planId	string (uuid)	
note	string	

200 OK → Subscription
400 401 403 404 429 standard errors

POST `/platform/sales-enquiries/{id}/create-company` Create Enterprise company `(platform: owner)`

Sets up a company sold through sales and emails the Admin an invitation.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`tenantType` *	`brokerage` | `individual`	
`companyName` *	string	
`tradeLicenseNo`	string	
`reraOrn`	string	
`adminEmail` *	string	
`adminFullName` *	string	
`seats` *	integer	
`pricePerSeat` *	string	Decimal as string, 2 places
`billingCycle` *	`monthly` | `yearly`	
`paymentMethod` *	`card` | `bank_transfer`	
`trialDays`	integer	

201 OK → PlatformTenant
400 401 403 404 429 standard errors

GET `/platform/sales-enquiries` List sales enquiries `(platform: owner, support)`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`status`	query	`new` | `contacted` | `won` | `lost`	

200 OK → { data: SalesEnquiry[], page: PageInfo }
400 401 403 429 standard errors

PATCH `/platform/sales-enquiries/{id}` Update sales enquiry `(platform: owner, support)`

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`status`	`new` | `contacted` | `won` | `lost`	
`note`	string	

200 OK → SalesEnquiry
400 401 403 404 429 standard errors

GET `/platform/billing-invoices` List all subscription invoices `(platform: owner, billing)`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	
`tenantId`	query	string (uuid)	
`status`	query	`open` | `paid` | `void` | `uncollectible`	
`from`	query	string (date)	
`to`	query	string (date)	

200 OK → { data: BillingInvoice[], page: PageInfo }
400 401 403 429 standard errors

POST `/platform/billing-invoices/{id}/void` Void subscription invoice `(platform: owner, billing)`

Only unpaid invoices. A corrected invoice is issued instead of editing.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`reason` *	string	

200 OK → BillingInvoice
400 401 403 404 409 429 standard errors

GET `/platform/bank-transfers` List pending bank transfers `(platform: owner, billing)`

PARAMETER	IN	TYPE	NOTES
`cursor`	query	string	Opaque cursor from page.nextCursor
`limit`	query	integer	

200 OK → { data: BillingPayment[], page: PageInfo }
400 401 403 429 standard errors

POST `/platform/billing-payments/{id}/confirm` Confirm bank transfer `(platform: owner, billing)`

Marks the invoice paid; a read-only or past-due company is reactivated at once.

PARAMETER	IN	TYPE	NOTES
`id` *	path	string (uuid)	

FIELD	TYPE	NOTES
`receivedAt` *	string (date-time)	
`amount`	string	Decimal as string, 2 places

200 OK → BillingPayment
400 401 403 404 429 standard errors

POST `/platform/billing-payments/{id}/reject` Reject bank transfer (platform: owner, billing)

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

FIELD	TYPE	NOTES
reason *	string	

200 OK → BillingPayment
400 401 403 404 429 standard errors

GET `/platform/plans` List all plans (platform: any)

200 OK → { data: Plan[] }
400 401 403 429 standard errors

POST `/platform/plans` Create plan (platform: owner)

Body: Plan

201 OK → Plan
400 401 403 409 429 standard errors

PATCH `/platform/plans/{id}` Update plan (platform: owner)

Price changes apply to new subscriptions; existing customers keep their agreed price until changed.

PARAMETER	IN	TYPE	NOTES
id *	path	string (uuid)	

Body: Plan

200 OK → Plan
400 401 403 404 429 standard errors

GET `/platform/settings` List platform settings (platform: any)

billing.trial (14 days, card required), billing.grace (14 days), billing.retention (90 days), billing.vat (5%), billing.payment_methods.

200 OK → { data: Setting[] }
400 401 403 429 standard errors

PUT `/platform/settings/{key}` Update platform setting (platform: owner)

PARAMETER	IN	TYPE	NOTES
key *	path	string	

FIELD	TYPE	NOTES
value *	object	

200 OK → Setting
400 401 403 404 429 standard errors

GET `/platform/audit-logs` List owner console audit log (platform: owner)

PARAMETER	IN	TYPE	NOTES
cursor	query	string	Opaque cursor from page.nextCursor
limit	query	integer	
tenantId	query	string (uuid)	
from	query	string (date)	
to	query	string (date)	

200 OK → { data: AuditEntry[], page: PageInfo }
400 401 403 429 standard errors

Webhooks (inbound)

GET `/webhooks/meta` Verify Meta webhook public

PARAMETER	IN	TYPE	NOTES
hub.mode	query	string	
hub.verify_token	query	string	
hub.challenge	query	string	

200 Echoes hub.challenge → string
400 429 standard errors

POST `/webhooks/meta` Receive Meta lead public

Signature: X-Hub-Signature-256. Stored, acknowledged in under 1 s, processed by the worker.

Body: object

200 OK → WebhookAck
400 401 429 standard errors

GET `/webhooks/whatsapp` Verify WhatsApp webhook public

PARAMETER	IN	TYPE	NOTES
hub.mode	query	string	
hub.verify_token	query	string	
hub.challenge	query	string	

200 Echoes hub.challenge → string
400 429 standard errors

POST `/webhooks/whatsapp` Receive WhatsApp message or status — public

Inbound messages, delivery statuses and template approvals. Signature: X-Hub-Signature-256.

Body: object

200 OK → WebhookAck
400 401 429 standard errors

POST `/webhooks/portals/{provider}` Receive portal lead — public

provider: property_finder, bayut, dubizzle. Verified with the shared secret set up for the account.

PARAMETER	IN	TYPE	NOTES
`provider` *	path	string	

Body: object

200 OK → WebhookAck
400 401 404 429 standard errors

POST `/webhooks/inbound/{token}` Receive general webhook lead — public

Any system (website, Google Sheets, Excel/Power Automate, Make.com). Header X-Signature: HMAC-SHA256 of the body with the source secret. Fields are mapped by the source's field mapping.

PARAMETER	IN	TYPE	NOTES
`token` *	path	string	

Body: object

200 OK → WebhookAck
400 401 404 429 standard errors

POST `/webhooks/email-inbound` Receive portal lead email — public

AWS SES → SNS notification for lead emails from portals; SNS signature verified.

Body: object

200 OK → WebhookAck
400 401 429 standard errors

POST `/webhooks/calendar/{provider}` Receive calendar change — public

provider: google, microsoft. Channel token verified.

PARAMETER	IN	TYPE	NOTES
`provider` *	path	string	

Body: object

200 OK → WebhookAck
400 401 404 429 standard errors

POST `/webhooks/stripe` Receive Stripe billing event — public

Card saved, invoice paid, payment failed, subscription updated. Signature: Stripe-Signature. Stored first, then processed by the worker: payment failed → past_due with a 14-day grace; still unpaid → read_only; paid → active.

Body: object

200 OK → WebhookAck
400 401 429 standard errors

Data models

PageInfo

FIELD	TYPE	NOTES
`nextCursor`	string | null	
`hasMore` *	boolean	
`total`	integer	Present only when total=true is requested

Problem

Error body (RFC 9457 problem details)

FIELD	TYPE	NOTES
`type`	string	
`title` *	string	
`status` *	integer	
`code` *	string	Stable machine code, e.g. LEAD_EXISTS
`detail`	string	
`errors`	object[]	
`requestId`	string	

IdRef

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name`	string	

Me

FIELD	TYPE	NOTES
`user` *	User	
`tenant` *	Tenant	
`permissions` *	string[]	
`dataScope` *	`own` | `team` | `division` | `company`	

Tenant

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`type` *	`brokerage` | `individual`	
`name` *	string	
`legalName`	string	
`tradeLicenseNo`	string	
`reraOrn`	string	
`reraBrn`	string	
`tradeLicenseFileId`	string (uuid) | null	
`verificationStatus` *	`pending` | `verified` | `rejected`	
`trn`	string	
`email`	string	
`phone`	string	
`address`	string	
`website`	string	
`logoFileId`	string (uuid) | null	
`timezone` *	string	
`baseCurrency` *	string	

User

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`email` *	string (email)	
`fullName` *	string	
`phone`	string | null	
`roleId` *	string (uuid)	
`teamId`	string (uuid) | null	
`status` *	`invited` | `active` | `inactive`	
`receivesLeads` *	boolean	
`languages`	string[]	
`location`	string | null	
`timezone`	string	
`lastLoginAt`	string (date-time) | null	

UserCreate

FIELD	TYPE	NOTES
`email` *	string (email)	
`fullName` *	string	
`phone`	string	
`roleId` *	string (uuid)	
`teamId`	string (uuid)	
`languages`	string[]	
`location`	string	

Role

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`key` *	string	
`name` *	string	
`dataScope` *	`own` | `team` | `division` | `company`	
`isSystem`	boolean	
`permissions` *	string[]	

Permission

FIELD	TYPE	NOTES
`key` *	string	
`module` *	string	
`description` *	string	

Team

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`divisionId`	string (uuid) | null	
`salesManagerUserId`	string (uuid) | null	
`teamLeaderUserId`	string (uuid) | null	
`isActive`	boolean	

Division

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`gmUserId`	string (uuid) | null	
`isActive`	boolean	

Setting

FIELD	TYPE	NOTES
`key` *	string	
`value` *	object	
`updatedAt`	string (date-time)	

LeadStage

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`key` *	string	
`name` *	string	
`category` *	`open` | `won` | `lost` | `invalid`	
`sortOrder` *	integer	
`isSystem`	boolean	
`isActive`	boolean	

Reason

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`type` *	`lost` | `invalid` | `reassignment`	
`key`	string | null	
`name` *	string	
`isActive`	boolean	

LeadSource

FIELD	TYPE	NOTES
id *	string (uuid)	
type *	meta | google_ads | linkedin | snapchat | tiktok | website | webhook | property_finder | bayut | dubizzle | whatsapp | import | manual | event | referral | other	
name *	string	
integrationId	string (uuid) | null	
qualityScore	integer	
isActive	boolean	

LeadCreate

At least one of phonePrimary or email is required.

FIELD	TYPE	NOTES
fullName *	string	
phonePrimary	string	
phoneAlternate	string	
email	string (email)	
countryCode	string	
preferredLanguage	string	
budgetMin	string	Decimal as string, 2 places
budgetMax	string	Decimal as string, 2 places
budgetCurrency	string	
sourceId *	string (uuid)	
campaignId	string (uuid)	
eventId	string (uuid)	
priority	low | medium | high	
projectIds	string (uuid)[]	
primaryProjectId	string (uuid)	
ownerUserId	string (uuid)	Optional. If empty, assignment rules choose the owner.
note	string	

LeadUpdate

FIELD	TYPE	NOTES
`fullName`	string	
`phonePrimary`	string	
`phoneAlternate`	string	
`email`	string (email)	
`countryCode`	string	
`preferredLanguage`	string	
`budgetMin`	string	Decimal as string, 2 places
`budgetMax`	string	Decimal as string, 2 places
`budgetCurrency`	string	
`campaignId`	string (uuid)	
`eventId`	string (uuid)	
`priority`	`low` | `medium` | `high`	
`projectIds`	string (uuid)[]	
`primaryProjectId`	string (uuid)	
`nextFollowUpAt`	string (date-time)	
`version` *	integer	Optimistic lock: send the version you read

Lead

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`fullName` *	string	
`phonePrimary`	string	
`phoneAlternate`	string	
`email`	string (email)	
`countryCode`	string	
`preferredLanguage`	string	
`budgetMin`	string	Decimal as string, 2 places
`budgetMax`	string	Decimal as string, 2 places
`budgetCurrency`	string	
`sourceId` *	string (uuid)	
`campaignId`	string (uuid)	
`eventId`	string (uuid)	
`priority`	`low` | `medium` | `high`	
`projectIds`	string (uuid)[]	
`primaryProjectId`	string (uuid)	
`ownerUserId`	string (uuid) | null	
`teamId`	string (uuid) | null	
`stageId` *	string (uuid)	
`stageCategory` *	`open` | `won` | `lost` | `invalid`	
`closeReasonId`	string (uuid) | null	
`score` *	integer	
`latestNote`	string | null	
`nextFollowUpAt`	string (date-time) | null	
`responseDueAt`	string (date-time) | null	
`enquiryCount` *	integer	
`lastEnquiryAt`	string (date-time)	
`access`	`owner` | `edit` | `view` | `scope`	How the current user can access this lead
`deletedAt`	string (date-time) | null	
`createdAt` *	string (date-time)	
`updatedAt`	string (date-time)	
`version` *	integer	

LeadExists

Returned with 409 LEAD_EXISTS when the phone or email already belongs to a lead

FIELD	TYPE	NOTES
`leadId` *	string (uuid)	
`ownerUserId`	string (uuid) | null	
`ownerName`	string | null	
`stageName`	string	

StageChange

reasonId is required when the stage category is lost or invalid

FIELD	TYPE	NOTES
`stageId` *	string (uuid)	
`reasonId`	string (uuid)	
`note`	string	
`nextFollowUpAt`	string (date-time)	

Assign

reasonId is required when the lead already has an owner

FIELD	TYPE	NOTES
`toUserId` *	string (uuid)	
`reasonId`	string (uuid)	
`note`	string	

Share

FIELD	TYPE	NOTES
`userId` *	string (uuid)	
`access` *	`view` | `edit`	

LeadShare

FIELD	TYPE	NOTES
`userId` *	string (uuid)	
`userName`	string	
`access` *	`view` | `edit`	
`sharedBy` *	string (uuid)	
`sharedAt` *	string (date-time)	

Note

FIELD	TYPE	NOTES
`body` *	string	

Activity

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`type` *	`inquiry` | `re_enquiry` | `note` | `call` | `stage_change` | `owner_change` | `share` | `whatsapp` | `email` | `meeting` | `viewing` | `attachment` | `deal` | `eoi` | `booking` | `invoice` | `purchase_order` | `payment` | `system`	
`summary` *	string	
`body`	string | null	
`actorUserId`	string (uuid) | null	
`occurredAt` *	string (date-time)	
`links`	object	

Inquiry

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`sourceId` *	string (uuid)	
`campaignId`	string (uuid) | null	
`isReEnquiry` *	boolean	
`externalCampaignId`	string | null	
`externalAdsetId`	string | null	
`externalAdId`	string | null	
`externalFormId`	string | null	
`externalSubmissionId`	string | null	
`formName`	string | null	
`formAnswers` *	object	
`submittedAt` *	string (date-time)	
`receivedAt` *	string (date-time)	

BulkLeadAction

FIELD	TYPE	NOTES
`leadIds` *	string (uuid)[]	
`action` *	`assign` | `stage` | `delete` | `restore`	
`toUserId`	string (uuid)	
`stageId`	string (uuid)	
`reasonId`	string (uuid)	

BulkResult

FIELD	TYPE	NOTES
`succeeded` *	string (uuid)[]	
`failed` *	object[]	

LeadImport

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`fileId` *	string (uuid)	
`mapping`	object	CRM field → column header
`sourceId`	string (uuid)	
`status` *	`previewing` | `importing` | `completed` | `failed`	
`preview`	object[]	
`rowsTotal`	integer	
`rowsImported`	integer	
`rowsReEnquired`	integer	
`rowsRejected`	integer	
`rejectedRowsFileId`	string (uuid) | null	

SavedFilter

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`entity` *	`leads` | `tasks` | `units` | `finance_documents` | `messages` | `attendance`	
`name` *	string	
`filters` *	object	
`columns`	string[]	
`sort`	string[]	
`sharedTeamId`	string (uuid) | null	

AssignmentRule

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`priority` *	integer	
`conditions` *	object	
`targetTeamId`	string (uuid) | null	
`memberUserIds`	string (uuid)[]	
`respectAvailability`	boolean	
`isActive`	boolean	

Campaign

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`sourceId` *	string (uuid)	
`externalId`	string | null	
`name` *	string	
`status` *	`active` | `paused` | `ended`	
`startsOn`	string (date) | null	
`endsOn`	string (date) | null	

AdSpend

FIELD	TYPE	NOTES
`spendDate` *	string (date)	
`amount` *	string	Decimal as string, 2 places
`currency` *	string	
`entryMethod`	`api` | `manual`	

Event

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`type` *	`event` | `roadshow` | `exhibition` | `open_house`	
`location`	string	
`startsAt` *	string (date-time)	
`endsAt` *	string (date-time)	
`cost`	string | null	
`currency`	string | null	

Task

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`leadId`	string (uuid) | null	
`assignedUserId` *	string (uuid)	
`type` *	`call` | `follow_up` | `meeting` | `viewing` | `personal`	
`title` *	string	
`description`	string | null	
`dueAt` *	string (date-time)	
`durationMinutes`	integer | null	
`reminderMinutes`	integer | null	
`meetingId`	string (uuid) | null	
`status` *	`open` | `completed` | `cancelled`	
`outcome`	`answered` | `no_answer` | `busy` | `wrong_number` | `call_back` | null	
`outcomeNote`	string | null	
`completedAt`	string (date-time) | null	

TaskComplete

outcome is required for call tasks; nextAction is required for lead tasks unless the lead is closed

FIELD	TYPE	NOTES
`outcome`	`answered` | `no_answer` | `busy` | `wrong_number` | `call_back`	
`outcomeNote`	string	
`nextAction`	object	

Meeting

FIELD	TYPE	NOTES
id *	string (uuid)	
leadId *	string (uuid)	
organizerUserId *	string (uuid)	
type *	meeting | viewing	
channel *	google_meet | teams | in_person | phone	
bookedVia	agent | booking_page	
joinUrl	string | null	
location	string | null	
projectId	string (uuid) | null	
unitId	string (uuid) | null	
dealId	string (uuid) | null	
startsAt *	string (date-time)	
endsAt *	string (date-time)	
clientTimezone	string	
status *	scheduled | completed | cancelled | no_show	
cancelReason	string | null	

MeetingCreate

409 AGENT_BUSY when the agent already has a meeting at that time

FIELD	TYPE	NOTES
leadId *	string (uuid)	
organizerUserId *	string (uuid)	
type *	meeting | viewing	
channel *	google_meet | teams | in_person | phone	
location	string	
projectId	string (uuid)	
unitId	string (uuid)	
dealId	string (uuid)	
startsAt *	string (date-time)	
endsAt *	string (date-time)	
clientTimezone	string	
sendConfirmation	boolean	

BookingPage

FIELD	TYPE	NOTES
`id`	string (uuid)	
`slug` *	string	
`title` *	string	
`meetingChannel`	`google_meet` | `teams` | `in_person` | `phone`	
`durationMinutes` *	integer	
`bufferMinutes`	integer	
`minNoticeHours`	integer	
`weeklyHours` *	object	
`isActive` *	boolean	
`publicUrl`	string	

PublicBookingPage

FIELD	TYPE	NOTES
`title` *	string	
`agentName` *	string	
`companyName`	string	
`durationMinutes`	integer	
`meetingChannel`	string	
`slots` *	object[]	

PublicBookingRequest

FIELD	TYPE	NOTES
`startsAt` *	string (date-time)	
`fullName` *	string	
`phone` *	string	
`email`	string (email)	
`timezone` *	string	
`message`	string	
`consentWhatsApp`	boolean	

PublicBookingConfirmation

FIELD	TYPE	NOTES
`startsAt` *	string (date-time)	
`endsAt` *	string (date-time)	
`joinUrl`	string | null	
`timezone` *	string	

Conversation

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`leadId` *	string (uuid)	
`channel` *	`whatsapp` | `email`	
`contactAddress` *	string	
`lastMessageAt`	string (date-time) | null	
`lastInboundAt`	string (date-time) | null	
`unreadCount` *	integer	
`replyWindowOpen`	boolean	WhatsApp: free-form reply allowed (24 h since last inbound)

Message

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`conversationId` *	string (uuid)	
`leadId` *	string (uuid)	
`direction` *	`inbound` | `outbound`	
`templateId`	string (uuid) | null	
`campaignId`	string (uuid) | null	
`subject`	string | null	
`body`	string | null	
`mediaFileId`	string (uuid) | null	
`status` *	`queued` | `sent` | `delivered` | `read` | `failed` | `received`	
`errorMessage`	string | null	
`sentBy`	string (uuid) | null	
`createdAt` *	string (date-time)	

MessageSend

WhatsApp outside the 24-hour window requires templateId. 422 OPTED_OUT if the lead opted out of the channel.

FIELD	TYPE	NOTES
`channel` *	`whatsapp` | `email`	
`templateId`	string (uuid)	
`variables`	object	
`subject`	string	
`body`	string	
`mediaFileId`	string (uuid)	

MessageTemplate

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`channel` *	`whatsapp` | `email`	
`name` *	string	
`language` *	string	
`category` *	`marketing` | `utility` | `authentication` | `transactional`	
`subject`	string | null	
`body` *	string	
`variables`	object[]	
`status` *	`draft` | `pending` | `approved` | `rejected` | `disabled`	

MessageCampaign

FIELD	TYPE	NOTES
id *	string (uuid)	
name *	string	
channel *	whatsapp | email	
templateId *	string (uuid)	
integrationId	string (uuid)	
audienceFilter *	object	
audienceCount	integer | null	
scheduledAt	string (date-time) | null	
status *	draft | pending_approval | approved | rejected | sending | completed | cancelled	
createdBy	string (uuid)	
approvedBy	string (uuid) | null	
rejectionNote	string | null	

CampaignReport

FIELD	TYPE	NOTES
audience *	integer	
skippedOptedOut	integer	
sent *	integer	
delivered *	integer	
failed *	integer	
replied *	integer	
optedOut *	integer	

Consent

FIELD	TYPE	NOTES
channel *	whatsapp | email | sms	
status *	opted_in | opted_out	
captureMethod *	form | import | reply | unsubscribe_link | manual | provider	
note	string | null	
recordedAt	string (date-time)	

Developer

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`trn`	string | null	
`contactName`	string | null	
`email`	string | null	
`phone`	string | null	
`address`	string | null	
`website`	string | null	

Project

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`developerId` *	string (uuid)	
`name` *	string	
`community`	string | null	
`city`	string	
`address`	string | null	
`latitude`	number | null	
`longitude`	number | null	
`description`	string | null	
`constructionStatus` *	`off_plan` | `under_construction` | `ready`	
`handoverDate`	string (date) | null	
`paymentPlan`	string | null	
`brochureFileId`	string (uuid) | null	
`isActive`	boolean	
`inventory`	object	

Building

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`projectId` *	string (uuid)	
`name` *	string	
`floors`	integer | null	
`handoverDate`	string (date) | null	

Unit

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`projectId` *	string (uuid)	
`buildingId`	string (uuid) | null	
`unitNumber` *	string	
`propertyType` *	`apartment` | `villa` | `townhouse` | `penthouse` | `duplex` | `office` | `retail` | `plot`	
`bedrooms` *	integer	
`bathrooms`	integer | null	
`sizeSqft` *	number	
`floor`	integer | null	
`view`	string | null	
`price` *	string	Decimal as string, 2 places
`currency` *	string	
`paymentPlan`	string | null	
`handoverDate`	string (date) | null	
`status` *	`available` | `on_hold` | `booked` | `sold`	
`holdExpiresAt`	string (date-time) | null	
`floorPlanFileId`	string (uuid) | null	

UnitStatusChange

Send status and/or price

FIELD	TYPE	NOTES
status	available | on_hold | booked | sold	
price	string	Decimal as string, 2 places
holdExpiresAt	string (date-time)	
reason	string	

UnitChange

FIELD	TYPE	NOTES
oldStatus	string | null	
newStatus	string | null	
oldPrice	string | null	
newPrice	string | null	
reason	string | null	
changedBy	string (uuid) | null	
changedAt *	string (date-time)	

Deal

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`leadId` *	string (uuid)	
`projectId` *	string (uuid)	
`unitId`	string (uuid) | null	
`ownerUserId` *	string (uuid)	
`stage` *	`offer` | `eoi` | `booking` | `sold` | `cancelled`	
`currency` *	string	
`offerPrice`	string | null	
`offerDate`	string (date) | null	
`eoiAmount`	string | null	
`eoiDate`	string (date) | null	
`bookingAmount`	string | null	
`bookingDate`	string (date) | null	
`saleValue`	string | null	
`saleDate`	string (date) | null	
`cancelledReason`	string | null	

DealStageChange

409 UNIT_ALREADY_BOOKED when another deal holds the unit

FIELD	TYPE	NOTES
`stage` *	`eoi` | `booking` | `sold` | `cancelled`	
`unitId`	string (uuid)	
`amount`	string	Decimal as string, 2 places
`date`	string (date)	
`cancelledReason`	string	

Counterparty

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`type` *	`client` | `developer` | `vendor`	
`name` *	string	
`trn`	string | null	
`email`	string | null	
`phone`	string | null	
`address`	string | null	
`leadId`	string (uuid) | null	
`developerId`	string (uuid) | null	

DocumentLine

FIELD	TYPE	NOTES
`lineNo` *	integer	
`description` *	string	
`quantity` *	string	
`unitPrice` *	string	Decimal as string, 2 places
`taxRate` *	string	
`lineSubtotal`	string	Decimal as string, 2 places
`taxAmount`	string	Decimal as string, 2 places

FinanceDocument

FIELD	TYPE	NOTES
id *	string (uuid)	
docType *	sales_invoice | purchase_invoice | purchase_order	
number	string | null	
counterpartyId *	string (uuid)	
leadId	string (uuid) | null	
projectId	string (uuid) | null	
unitId	string (uuid) | null	
dealId	string (uuid) | null	
purchaseOrderId	string (uuid) | null	
issueDate *	string (date)	
dueDate	string (date) | null	
currency *	string	
rateToBase	string	
lines *	DocumentLine[]	
subtotal	string	Decimal as string, 2 places
taxTotal	string	Decimal as string, 2 places
total	string	Decimal as string, 2 places
amountPaid	string	Decimal as string, 2 places
amountRefunded	string	Decimal as string, 2 places
balance	string	Decimal as string, 2 places
paymentTerms	string | null	
notes	string | null	
status *	draft | approved | sent | partially_paid | paid | overdue | cancelled | refunded	
pdfFileId	string (uuid) | null	
version *	integer	

Payment

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`kind` *	`payment` | `refund`	
`amount` *	string	Decimal as string, 2 places
`currency` *	string	
`paidOn` *	string (date)	
`method` *	`bank_transfer` | `cheque` | `card` | `cash` | `other`	
`reference`	string | null	
`notes`	string | null	
`proofFileId`	string (uuid) | null	

DocumentEvent

FIELD	TYPE	NOTES
`action` *	string	
`fromStatus`	string | null	
`toStatus`	string | null	
`actorUserId`	string (uuid) | null	
`note`	string | null	
`createdAt` *	string (date-time)	

Commission

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`dealId` *	string (uuid)	
`userId` *	string (uuid)	
`basis` *	`percentage` | `fixed`	
`rate`	string | null	
`amount` *	string	Decimal as string, 2 places
`currency` *	string	
`status` *	`pending` | `approved` | `paid` | `cancelled`	
`developerInvoiceId`	string (uuid) | null	
`paidOn`	string (date) | null	
`notes`	string | null	

CancelRequest

FIELD	TYPE	NOTES
`reason` *	string	

CheckIn

Location is required only when the tenant's policy requires it

FIELD	TYPE	NOTES
`photoFileId` *	string (uuid)	
`latitude`	number	
`longitude`	number	
`workFromHome`	boolean	

AttendanceRecord

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`userId` *	string (uuid)	
`workDate` *	string (date)	
`status` *	`present` | `late` | `absent` | `leave` | `half_day` | `wfh`	
`checkInAt`	string (date-time) | null	
`checkOutAt`	string (date-time) | null	
`checkInPhotoFileId`	string (uuid) | null	
`checkOutPhotoFileId`	string (uuid) | null	
`isCorrected` *	boolean	

AttendanceCorrection

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`recordId` *	string (uuid)	
`requestedStatus` *	`present` | `late` | `absent` | `leave` | `half_day` | `wfh`	
`requestedCheckInAt`	string (date-time) | null	
`requestedCheckOutAt`	string (date-time) | null	
`reason` *	string	
`status` *	`pending` | `approved` | `rejected`	
`originalValues`	object	
`reviewedBy`	string (uuid) | null	
`reviewNote`	string | null	

LeaveRequest

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`userId` *	string (uuid)	
`leaveTypeId` *	string (uuid)	
`startDate` *	string (date)	
`endDate` *	string (date)	
`isHalfDay`	boolean	
`reason`	string | null	
`status` *	`pending` | `approved` | `rejected` | `cancelled`	
`reviewedBy`	string (uuid) | null	
`reviewNote`	string | null	

Review

FIELD	TYPE	NOTES
`decision` *	`approve` | `reject`	
`note`	string	

AttendancePolicy

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`workStart` *	string	
`workEnd` *	string	
`graceMinutes`	integer	
`halfDayMaxMinutes`	integer	
`weekendDays` *	integer[]	
`locationRequired`	boolean	
`officeLatitude`	number | null	
`officeLongitude`	number | null	
`officeRadiusM`	integer | null	
`isDefault`	boolean	

Holiday

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`holidayDate` *	string (date)	
`name` *	string	

LeaveType

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`annualDays`	string | null	
`isPaid`	boolean	
`isActive`	boolean	

ReportDefinition

FIELD	TYPE	NOTES
`key` *	string	
`name` *	string	
`module`	string	
`filters` *	string[]	
`columns` *	object[]	

ReportResult

FIELD	TYPE	NOTES
`columns` *	object[]	
`rows` *	object[]	
`totals` *	object	
`page`	PageInfo	

ExportRequest

FIELD	TYPE	NOTES
`format` *	`xlsx` | `csv`	
`filters`	object	

Export

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`reportKey` *	string	
`format` *	`xlsx` | `csv`	
`status` *	`queued` | `running` | `completed` | `failed`	
`rowCount`	integer | null	
`downloadUrl`	string | null	Time-limited link, present when completed
`createdAt` *	string (date-time)	

Dashboard

FIELD	TYPE	NOTES
`kpis` *	object	
`bySource`	CountBucket[]	
`byProject`	CountBucket[]	
`byStage`	CountBucket[]	
`funnel` *	CountBucket[]	
`byAgent`	CountBucket[]	
`byCampaign`	CountBucket[]	
`trend`	object[]	
`refreshedAt` *	string (date-time)	

CountBucket

FIELD	TYPE	NOTES
`key` *	string	
`label` *	string	
`count` *	integer	
`value`	string | null	
`drillDown`	object	Filters that open the underlying records

Integration

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`provider` *	`meta` | `google_ads` | `linkedin` | `snapchat` | `tiktok` | `website` | `webhook` | `property_finder` | `bayut` | `dubizzle` | `whatsapp_cloud` | `email_ses` | `gmail` | `microsoft_365` | `google_workspace`	
`scope` *	`tenant` | `user`	
`userId`	string (uuid) | null	
`accountLabel` *	string	
`status` *	`pending` | `connected` | `error` | `disconnected`	
`lastSuccessAt`	string (date-time) | null	
`lastErrorAt`	string (date-time) | null	
`lastError`	string | null	
`recordsReceived30d`	integer	
`failures30d`	integer	
`settings`	object	

IntegrationConnect

FIELD	TYPE	NOTES
`provider` *	string	
`scope`	`tenant` | `user`	
`accountLabel`	string	
`settings`	object	

ConnectStart

FIELD	TYPE	NOTES
`integrationId` *	string (uuid)	
`authorizeUrl`	string | null	

WebhookSource

FIELD	TYPE	NOTES
`integrationId`	string (uuid)	
`name` *	string	
`inboundUrl`	string	
`secret`	string	Shown once; used to sign requests (HMAC-SHA256)
`sourceId` *	string (uuid)	
`fieldMapping` *	object	

SyncRun

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`jobType` *	string	
`status` *	`running` | `success` | `partial` | `failed`	
`recordsReceived`	integer	
`recordsCreated`	integer	
`recordsFailed`	integer	
`errorDetail`	string | null	
`startedAt` *	string (date-time)	
`finishedAt`	string (date-time) | null	

Reconciliation

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`integrationId` *	string (uuid)	
`periodStart` *	string (date-time)	
`periodEnd` *	string (date-time)	
`sourceCount`	integer	
`crmCount`	integer	
`missingExternalIds`	string[]	
`recoveredCount`	integer	
`status` *	`running` | `completed` | `failed`	

UploadRequest

FIELD	TYPE	NOTES
`fileName` *	string	
`mimeType` *	string	
`sizeBytes` *	integer	
`purpose` *	`lead_attachment` | `project_media` | `unit_floor_plan` | `finance` | `attendance_photo` | `import` | `message_media` | `logo`	

UploadTicket

FIELD	TYPE	NOTES
`fileId` *	string (uuid)	
`uploadUrl` *	string	PUT the file bytes here within 10 minutes
`headers`	object	

DownloadLink

FIELD	TYPE	NOTES
`url` *	string	
`expiresAt` *	string (date-time)	

Attachment

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`fileId` *	string (uuid)	
`fileName`	string	
`category` *	`document` | `image` | `brochure` | `floor_plan` | `contract` | `id_document` | `receipt` | `other`	
`createdAt`	string (date-time)	

Notification

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`type` *	string	
`title` *	string	
`body`	string | null	
`leadId`	string (uuid) | null	
`linkPath`	string | null	
`readAt`	string (date-time) | null	
`createdAt` *	string (date-time)	

PushDevice

FIELD	TYPE	NOTES
`fcmToken` *	string	
`platform` *	`android` | `ios` | `web`	

AuditEntry

FIELD	TYPE	NOTES
`id` *	integer	
`userId`	string (uuid) | null	
`action` *	string	
`entityType` *	string	
`entityId`	string (uuid) | null	
`changes`	object | null	
`ipAddress`	string | null	
`createdAt` *	string (date-time)	

WebhookAck

FIELD	TYPE	NOTES
`received` *	boolean	

SearchResult

FIELD	TYPE	NOTES
`leads` *	IdRef[]	
`projects`	IdRef[]	
`units`	IdRef[]	
`users`	IdRef[]	

CallLog

Log a call that was not planned as a task

FIELD	TYPE	NOTES
`outcome` *	`answered` | `no_answer` | `busy` | `wrong_number` | `call_back`	
`note`	string	
`durationSeconds`	integer	
`nextAction`	object	

WebhookEvent

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`provider` *	string	
`externalEventId`	string	
`status` *	`received` | `processed` | `failed` | `ignored`	
`attempts` *	integer	
`lastError`	string | null	
`receivedAt` *	string (date-time)	
`processedAt`	string (date-time) | null	

Plan

Prices are per user, excluding 5% VAT

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`key` *	string	
`name` *	string	
`description`	string | null	
`tenantType`	`brokerage` | `individual` | null	
`priceMonthly` *	string	Decimal as string, 2 places
`priceYearly` *	string	Decimal as string, 2 places
`currency` *	string	
`minSeats` *	integer	
`maxSeats`	integer | null	
`selfService` *	boolean	
`isActive`	boolean	

Subscription

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`plan` *	Plan	
`status` *	`trialing` | `active` | `past_due` | `read_only` | `canceled`	
`billingCycle` *	`monthly` | `yearly`	
`seats` *	integer	
`seatsUsed` *	integer	Active users now
`pricePerSeat` *	string	Decimal as string, 2 places
`currency` *	string	
`paymentMethod` *	`card` | `bank_transfer`	
`cardLast4`	string | null	
`trialEndsAt`	string (date-time) | null	
`currentPeriodEnd`	string (date-time) | null	
`graceEndsAt`	string (date-time) | null	
`cancelAtPeriodEnd` *	boolean	
`dataDeleteAfter`	string (date) | null	
`nextInvoiceEstimate`	object	

BillingInvoice

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`number` *	string	
`status` *	`open` | `paid` | `void` | `uncollectible`	
`periodStart` *	string (date)	
`periodEnd` *	string (date)	
`issueDate` *	string (date)	
`dueDate` *	string (date)	
`currency` *	string	
`lines`	object[]	
`subtotal` *	string	Decimal as string, 2 places
`vatRate`	string	
`vatAmount` *	string	Decimal as string, 2 places
`total` *	string	Decimal as string, 2 places
`amountPaid`	string	Decimal as string, 2 places
`balance` *	string	Decimal as string, 2 places
`paidAt`	string (date-time) | null	

BillingPayment

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`invoiceId` *	string (uuid)	
`invoiceNumber`	string	
`tenantId`	string (uuid)	
`tenantName`	string	
`method` *	`card` | `bank_transfer`	
`status` *	`pending` | `succeeded` | `failed` | `refunded`	
`amount` *	string	Decimal as string, 2 places
`currency` *	string	
`bankReference`	string | null	
`proofFileId`	string (uuid) | null	
`failureReason`	string | null	
`receivedAt`	string (date-time) | null	
`createdAt` *	string (date-time)	

SubscriptionEvent

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`type` *	string	
`oldValue`	object | null	
`newValue`	object | null	
`actorType` *	`user` | `platform_admin` | `system` | `stripe`	
`actorName`	string | null	
`note`	string | null	
`createdAt` *	string (date-time)	

CardSetup

Used by Stripe Elements in the browser; card details never reach our servers

FIELD	TYPE	NOTES
`stripePublishableKey` *	string	
`setupIntentClientSecret` *	string	

SignupStart

Brokerage: tradeLicenseNo and reraOrn required. Individual agent: reraBrn required.

FIELD	TYPE	NOTES
`tenantType` *	`brokerage` | `individual`	
`companyName` *	string	
`tradeLicenseNo`	string	
`reraOrn`	string	
`reraBrn`	string	
`adminFullName` *	string	
`adminEmail` *	string (email)	
`adminPhone` *	string	
`planKey` *	string	
`seats`	integer	
`billingCycle` *	`monthly` | `yearly`	
`acceptTerms` *	boolean	
`turnstileToken` *	string	

Signup

FIELD	TYPE	NOTES
`signupId` *	string (uuid)	
`emailVerified` *	boolean	
`cardSetup` *	CardSetup	
`expiresAt` *	string (date-time)	

SignupComplete

FIELD	TYPE	NOTES
`tenantId` *	string (uuid)	
`trialEndsAt` *	string (date-time)	
`loginUrl` *	string	

SalesEnquiry

FIELD	TYPE	NOTES
`id`	string (uuid)	
`companyName` *	string	
`contactName` *	string	
`email` *	string (email)	
`phone`	string	
`usersEstimate`	integer	
`message`	string	
`status`	`new` | `contacted` | `won` | `lost`	
`createdAt`	string (date-time)	

BillingUsage

FIELD	TYPE	NOTES
`seats` *	integer	
`seatsUsed` *	integer	
`invitedUsers`	integer	
`leadsThisMonth`	integer	
`messagesThisMonth`	integer	
`storageBytes`	integer	

PlatformTenant

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`name` *	string	
`type` *	`brokerage` | `individual`	
`legalName`	string | null	
`tradeLicenseNo`	string | null	
`reraOrn`	string | null	
`reraBrn`	string | null	
`tradeLicenseFileId`	string (uuid) | null	
`verificationStatus` *	`pending` | `verified` | `rejected`	
`status` *	`active` | `suspended` | `closed`	
`subscription` *	Subscription	
`usage`	BillingUsage	
`adminEmail`	string	
`createdAt`	string (date-time)	
`lastActiveAt`	string (date-time) | null	

PlatformMetrics

FIELD	TYPE	NOTES
`mrr` *	string	Decimal as string, 2 places
`arr`	string	Decimal as string, 2 places
`payingTenants` *	integer	
`trialingTenants` *	integer	
`pastDueTenants`	integer	
`readOnlyTenants`	integer	
`paidSeats` *	integer	
`newTrials30d`	integer	
`trialConversion30d`	number	
`churned30d`	integer	
`pendingBankTransfers`	integer	
`pendingVerifications`	integer	

PlatformAdmin

FIELD	TYPE	NOTES
`id` *	string (uuid)	
`email` *	string	
`fullName` *	string	
`role` *	`owner` | `support` | `billing`	